A Project Report on

VISTA – Vision-Integrated Smart Traffic Automation

Submitted in partial fulfillment of the requirements for the award
of the degree of

Bachelor of Engineering

in

Computer Science and Engineering - Data Science

by

Sahil Gorde (22107035)
Siddhi Patwardhan (22107033)
Varad Chaudhari(22107053)
Kisankumar Jena(22107049)

Under the Guidance of

Ms. Sarala Mary
Mrs. Richa Singh

Department of Computer Science and Engineering - Data Science
A.P. Shah Institute of Technology
G.B.Road,Kasarvadavli, Thane(W)-400615
UNIVERSITY OF MUMBAI

Academic Year 2025-2026

Approval Sheet

This Project Report entitled ***"VISTA – Vision-Integrated Smart Traffic Automation"*** Submitted by ***"Sahil Gorde"(22107035),"Siddhi Patwardhan(22107033), "VaradChaudhari(22107053),"Kisankumar Jena "(22107049)***is approved for the partial fulfillment of the requirement for the award of the degree of ***Bachelor of Engineering*** in ***Computer Science and Engineering - Data Science*** from ***University of Mumbai***.

(Mrs. Richa Singh)
Co-Guide

(Ms. Sarala Mary)
Guide

Dr. Pravin Adivarekar
HOD, Computer Science and Engineering - Data Science

Place:A.P.Shah Institute of Technology, Thane
Date:

CERTIFICATE

This is to certify that the project entitled ***"VISTA – Vision-Integrated Smart Traffic Automation"*** submitted by ***"Sahil Gorde" (22107035),"Siddhi Patwardhan" (22107033),"Varad Chaudhari " (22107053),"Kisankumar Jena " (22107049)*** for the partial fulfillment of the requirement for award of a degree ***Bachelor of Engineering*** in ***Computer Science and Engineering - Data Science***,to the University of Mumbai,is a bonafide work carried out during academic year 2025-2026.

(Mrs. Richa Singh)
Co-Guide

(Ms. Sarala Mary)
Guide

Dr. Pravin Adivarekar
HOD, CSE(Data Science)

Dr. Uttam D.Kolekar
Principal

External Examiner(s)

1.

2.

Internal Examiner(s)

1.

2.

Place:A.P.Shah Institute of Technology, Thane
Date:

Acknowledgement

We have great pleasure in presenting the report on **VISTA – Vision-Integrated Smart Traffic Automation** We take this opportunity to express our sincere thanks towards our guide **Ms. Sarala Mary** & Co-Guide **Mrs. Richa Singh** for providing the technical guidelines and suggestions regarding line of work. We would like to express our gratitude towards his constant encouragement, support and guidance through the development of project.

We thank **Dr. Pravin Adivarekar** Head of Department for his encouragement during the progress meeting and for providing guidelines to write this report.

We express our gratitude towards BE project co-ordinators, for being encouraging throughout the course and for their guidance.

We also thank the entire staff of APSIT for their invaluable help rendered during the course of this work. We wish to express our deep gratitude towards all our colleagues of APSIT for their encouragement.

Sahil Gorde
(22107035)

Siddhi Patwardhan
(22107033)

Varad Chaudhari
(22107053)

Kisankumar Jena
(22107049)

Declaration

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, We have adequately cited and referenced the original sources. We also declare that We have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(Signature)
Sahil Gorde(22107035)

(Signature)
Siddhi Patwardhan(22107033)

(Signature)
Varad Chaudhari(22107053)

(Signature)
Kisankumar Jena (22107049)

Date:

Abstract

Traffic signal control (TSC) is a major challenge in urban mobility, where realtime decisions must balance efficiency, safety, and fairness. Conventional systems—fixed-timed, sensor-based, or RL-driven—struggle with dynamic traffic, emergencies, and rule violations. We propose an AI-powered Smart Traffic Automation System that integrates computer vision, deep learning, and language-driven decision-making for adaptive control. YOLO-based perception analyzes live video streams to detect vehicles, pedestrians, emergency vehicles, and congestion for dynamic timing. A Large Language Model (LLM) framework acts as the intelligent decision controller for the system. The Perception Agent interprets real-time traffic scenes from camera inputs, while the Safety-Prioritized Meta Controller determines whether to invoke structured reasoning for emergencies or apply reinforcement learning for routine traffic flow and the Planning Agent finalizes the optimal signal timing and coordination based on traffic density and priority. This integrated combination of perception, reasoning, and adaptive decision-making creates a scalable, safety-aware, and intelligent traffic management solution for modern urban intersections.

Keywords - YOLO, Computer Vision, Reinforcement Learning(PPO), Large Language Models (LLMs), Perception Agent, Meta Controller, SUMO Simulation.

Contents

1 Introduction **1**
1.1 Motivation . . . 2
1.2 Problem Statement . . . 3
1.3 Objectives . . . 4
1.4 Scope . . . 5

2 Literature Review **6**

3 Project Design **12**
3.1 Existing System . . . 12
3.2 Proposed System . . . 14
3.2.1 Critical Components of System Architecture . . . 15
3.3 System Diagrams . . . 17
3.3.1 UML Diagram . . . 17
3.3.2 DataFlow Diagram . . . 17
3.3.3 Use Case Diagram . . . 18
3.3.4 Sequence Diagram . . . 20

4 Project Implementation **22**
4.1 Code Snippets . . . 22
4.2 Steps to access the System . . . 29
4.3 Timeline Sem VIII . . . 35

5 Testing **37**
5.1 Software Testing . . . 37
5.2 Functional Testing . . . 38

6 Result and Discussions **40**

7 Conclusion **45**

8 Future Scope **46**

Bibliography **48**

List of Figures

3.1 System Architecture of the Proposed Intelligent Traffic Management System 14
3.2 Critical Components of System Architecture 16
3.3 VISTA System Data Flow Diagram 18
3.4 VISTA System Use Case Diagram 19
3.5 VISTA System Sequence Diagram 20

4.1 PPO Policy Update Function 22
4.2 LLM agent Description Service 23
4.3 Emergency Handling Module: Direction Mapping and Duration Calculation . 24
4.4 Real-Time Accident Detection and Alert Logic 25
4.5 Red Light Violation Detection Logic 26
4.6 Pedestrian Safety Phase Implementation 27
4.7 Live Camera Feeds Interface. 29
4.8 Real-time Vehicle Counts and AI Analysis. 30
4.9 Meta Controller Analysis. 31
4.10 Meta Controller Analysis. 31
4.11 Professional Traffic Intersection Control. 32
4.12 Decision Log. 33
4.13 Crash Detection System. 33
4.14 Red Light Jump Detection. 34
4.15 Red Light Jump Detection. 35
4.16 Gantt chart showing project milestones. 36

6.1 Comparison of Policies Based on Reward and Queue Metrics 40
6.2 Comparative performance models based on queue length and trip time percentage 41
6.3 Precision–Recall Curve for Ambulance detection model 43
6.4 Precision–Recall Curve for Accident detection model 44

List of Tables

2.1 Comparative Analysis of Literature Survey 9

5.1 Functional Testing Table for VISTA - Part 1 38
5.2 Functional Testing Table for VISTA - Part 2 39

I Performance Metrics for Ambulance detection model 42
II Performance Metrics for Accident detection model 43

List of Abbreviations

VISTA:	Vision-Integrated Smart Traffic Automation
PPO:	Proximal Policy Optimization
LLM:	Large Language Model
RL:	Reinforcement Learning
YOLO:	You Only Look Once
OpenCV:	Open Source Computer Vision
R-CNN:	Region-Based Convolutional Neural Networks
DQN:	Deep Q-Networks
MARL:	Multi-Agent Reinforcement Learning
LSTM:	Long Short-Term Memory

Chapter 1

Introduction

In today's rapidly urbanizing world, traffic congestion has become a major concern affecting the daily lives of millions of commuters. With the continuous rise in the number of vehicles and limited expansion of road infrastructure, city intersections often experience long delays, fuel wastage, and increased air pollution. Managing such dynamic traffic situations through conventional signal systems has proven to be inefficient, as these systems follow fixed-time schedules that fail to adapt to real-time conditions on the road. In Indian urban junctions, the situation becomes more complex due to the mixed traffic of cars, bikes, pedestrians, and emergency vehicles, often sharing the same lanes without strict lane discipline. This makes adaptive, intelligent control far more necessary than fixed systems. The need for a smarter, more adaptive, and data-driven solution has therefore become essential for modern cities.

Traditional traffic control systems operate on predetermined signal cycles that do not consider actual vehicle density, sudden congestion, or emergency scenarios. As a result, vehicles are often made to wait unnecessarily, even when some lanes are empty, leading to poor traffic flow and frustration among road users. While some modern systems use sensors or timers to estimate traffic density, they still lack the intelligence to make context-aware decisions. These limitations have motivated researchers and city planners to explore the use of artificial intelligence to create systems that can perceive, analyze, and act autonomously.

The AI-Powered Smart Traffic Automation System proposed in this project aims to overcome these challenges by introducing an intelligent, adaptive approach to traffic signal management. The system uses artificial intelligence to monitor traffic conditions in real time and adjust the signal timings dynamically based on vehicle movement and density. Instead of relying on rigid rules, it learns from patterns and responds according to the current situation at the junction. This allows smoother traffic flow, reduced waiting time, and better utilization of available road space.

The project brings together three key components — perception, decision-making, and planning — to function as an integrated unit. The perception module observes traffic through live camera feeds and interprets the scene, identifying how many vehicles are present and in which directions. The decision-making component determines the best response depending on whether the situation is routine or critical, such as the presence of emergency vehicles. Finally, the planning unit decides the appropriate signal timing and coordination between

directions to ensure safety and efficiency.

By combining intelligent observation, reasoning, and adaptive control, the system represents a step forward in modern traffic management. It not only reduces congestion and improves travel time but also ensures that emergency vehicles and high-priority movements receive the attention they require. Ultimately, this project envisions a future where traffic signals think, adapt, and cooperate to create safer and more efficient roads for everyone.

1.1 Motivation

Traditional traffic signal systems in cities largely rely on fixed-time cycles or basic sensor-based mechanisms, which fail to adapt to the ever-changing dynamics of urban traffic. These outdated systems assume that traffic density is uniform and predictable, applying the same signal duration regardless of actual conditions on the road. In reality, however, intersections experience highly variable loads: one direction may be congested with long queues of vehicles, while another remains relatively empty. This mismatch leads to wasted green time, longer waiting periods, excessive fuel consumption, and higher emissions, all of which directly impact commuters' daily lives.

The challenge becomes even more critical in the Indian context, where mixed traffic consisting of cars, two-wheelers, buses, pedestrians, and emergency vehicles shares the same roads. Unlike systems abroad that follow strict lane discipline, Indian intersections often face irregular vehicle movement, making it difficult for traditional rule-based systems to ensure smooth flow. Moreover, critical scenarios—such as ambulances stuck in traffic or pedestrians stranded at busy crossings—are often neglected in existing systems, causing serious delays and safety risks.

Even advanced reinforcement learning–based traffic control systems developed in research often optimize for efficiency but overlook safety, fairness, and explainability. For example, while an RL agent may reduce overall congestion, it may fail to recognize the urgent need to prioritize an ambulance or provide safe pedestrian crossings. Similarly, traffic authorities often hesitate to trust "black-box" AI systems because they cannot explain their decisions in human terms.

These limitations highlight the need for a smarter, AI-driven solution that not only adapts to real-time traffic patterns but also balances efficiency, safety, and transparency. By integrating Proximal Policy Optimization (PPO) for adaptive timing and Large Language Model (LLM) agents for reasoning and emergency overrides, this project bridges the gap between raw optimization and human-centric decision-making. The system ensures smoother traffic flow, faster emergency response, and safer pedestrian movement, all while providing traffic authorities with explainable insights into its decisions.

Ultimately, the motivation behind this project lies in building a scalable and intelligent traffic management system that addresses the unique challenges of Indian urban intersections while paving the way for safer, greener, and smarter cities.

1.2 Problem Statement

In the age of rapid urbanization and smart city initiatives, traffic management remains one of the most persistent challenges faced by modern societies. Despite advances in infrastructure, the ever-increasing number of vehicles, coupled with limited road expansion, has led to severe congestion, longer travel times, and increased fuel consumption in urban areas. Traditional traffic signal systems, which rely on fixed-time cycles or basic sensor inputs, are unable to adapt to dynamic real-world conditions such as sudden congestion surges, accidents, or the unpredictable movement of pedestrians and emergency vehicles. This results in inefficient traffic flow, unnecessary delays, and a decline in road safety.

The complexity is further amplified in Indian traffic scenarios, where intersections handle a heterogeneous mix of cars, bikes, buses, auto-rickshaws, pedestrians, and emergency vehicles—all often moving without strict lane discipline. Existing systems are ill-equipped to deal with such irregular and high-density traffic conditions. As a consequence, vehicles in less congested lanes are forced to wait unnecessarily, while emergency vehicles like ambulances and fire trucks are frequently trapped in long queues, leading to delayed critical response times and avoidable risks to human life.

Even with the introduction of reinforcement learning (RL)–based traffic control models in research, several limitations persist. Many RL approaches optimize only for throughput or waiting time but fail to account for safety, fairness, and critical overrides. They lack the capacity to prioritize emergency situations, allocate pedestrian crossings effectively, or provide explainable decisions to traffic authorities. This lack of transparency makes it difficult for authorities to fully trust and deploy AI-driven systems in real-world environments.

Another significant gap lies in the absence of intelligent reasoning and adaptability. Rule-based overrides, when present, are rigid and unable to handle unforeseen scenarios such as sudden road blockages, VIP convoys, or coordinated traffic events. Without a reasoning layer, such systems remain "black boxes," offering little insight into why a particular signal decision was made. This reduces confidence among policymakers and limits large-scale adoption.

Given these challenges, there is an urgent need for a next-generation traffic management solution that combines the adaptive optimization power of reinforcement learning (specifically, Proximal Policy Optimization – PPO) with the context-aware reasoning and explainability of Large Language Models (LLMs). Such a system would not only dynamically adjust green light durations to maximize flow efficiency but also intelligently handle special cases like emergency vehicles, pedestrian crossings, and traffic violations. By providing human-readable reasoning for its decisions, the system can ensure greater trust, transparency, and acceptance by traffic authorities.

In conclusion, the problem this project addresses is the inadequacy of current traffic management systems—both traditional and AI-based—in dealing with the dynamic, heterogeneous, and safety-critical nature of real-world traffic, especially in the Indian context. The proposed solution aims to overcome these limitations by developing an AI-driven, adaptive,

and explainable traffic signal control system that ensures efficiency, safety, and fairness while being scalable for real-world deployment.

1.3 Objectives

The proposed AI-driven Traffic Signal Control System is designed to address the pressing challenges of congestion, safety, and fairness in modern urban traffic management, with a special focus on the complexities of Indian intersections. By combining reinforcement learning, computer vision, and large language model reasoning, the system aims to deliver a holistic and practical solution for improving traffic flow and prioritizing critical movements. The primary objectives of this project are described below.

- To dynamically optimize traffic signal timings using reinforcement learning (PPO) integrated with real-time vehicle detection: The system adapts green light durations based on live vehicle counts and traffic density captured by computer vision. By applying Proximal Policy Optimization (PPO), it learns optimal signal timing policies that minimize waiting times, reduce congestion, and improve throughput across intersections. Unlike fixed-time or sensor-only systems, this approach ensures adaptive control that responds effectively to fluctuating traffic patterns.

- To minimize delays for emergency vehicles through automated recognition and LLM-guided overrides: Emergency vehicles such as ambulances and fire trucks are detected in real-time using YOLO-based vision models. Once identified, the Safety-Prioritized Meta Controller powered by LLM reasoning overrides the normal PPO signal cycle, granting instant green clearance along the vehicle's route. This significantly reduces emergency response time and ensures life-saving priority handling.

- To enhance pedestrian safety with intelligent detection and dedicated crossing phases: Using YOLO-based pedestrian recognition, the system identifies pedestrians waiting at intersections and dynamically allocates safe crossing windows. This prevents conflicts between vehicles and vulnerable road users, reducing the chances of accidents while ensuring inclusivity for elderly, disabled, and school-going pedestrians.

- To automate traffic violation detection with computer vision and support enforcement through logged records: The system applies OpenCV and YOLO to detect common violations such as red-light jumping, lane misuse, or stop-line crossings. These violations are logged automatically and integrated with a backend for review by traffic authorities. This reduces reliance on manual monitoring and improves the efficiency of law enforcement.

1.4 Scope

The scope of this project defines the boundaries within which the Intelligent Traffic Signal Control System will operate. While traffic management in urban cities involves a wide range of issues such as poor infrastructure, lane discipline, and road encroachments, this project specifically focuses on signal optimization, emergency handling, pedestrian safety, and violation detection. By applying reinforcement learning, computer vision, and LLM-based reasoning, the system is designed to create a balance between efficiency, safety, and accountability at traffic intersections. The following points outline the major scope areas of the project:

- Dynamic Signal Optimization using PPO: The system adapts traffic signal timings in real time by applying Proximal Policy Optimization (PPO). Unlike static, pre-set cycles, the model learns from vehicle density and waiting times to minimize congestion and ensure smoother traffic flow across all directions.

- Emergency Vehicle Prioritization: YOLO-based detection models identify ambulances or fire trucks at intersections. Once detected, the Safety-Prioritized Meta Controller with LLM overrides the normal PPO cycle, granting instant green clearance. This significantly reduces delays for critical response vehicles, saving valuable time during emergencies.

- Pedestrian Safety through Intelligent Detection: Pedestrians waiting at intersections are recognized using computer vision. The system allocates safe and timely crossing phases, preventing conflicts with vehicle movements. This ensures inclusivity for vulnerable road users such as children, elderly citizens, and the differently-abled.

- Traffic Violation Detection and Logging: OpenCV and YOLO are employed to detect violations like red-light jumping, stop-line crossing, and lane misuse. These incidents are logged in real time and made available to traffic authorities for enforcement. This promotes accountability and discipline on the roads.

- Explainability and Trust via LLM Agents: The system integrates LLM-based reasoning agents to provide human-readable explanations for signal decisions. This enhances transparency, allowing traffic authorities to understand, audit, and trust the AI's decision-making process instead of treating it as a black box.

Chapter 2

Literature Review

The Literature Review explores various methodologies and frameworks that address challenges in intelligent traffic management, emergency vehicle detection, pedestrian recognition, and automated enforcement of traffic rules. The reviewed works include techniques such as deep learning-based computer vision for vehicle and pedestrian detection, AI-powered systems for real-time traffic monitoring, and large language model (LLM) integration for intelligent decision-making and alert generation. These studies highlight key limitations such as difficulties in accurately detecting emergency vehicles in complex traffic conditions, high computational requirements for real-time processing, and inefficiencies in integrating multiple data sources for automated traffic management. By analyzing these drawbacks, this review identifies gaps in existing solutions and provides a foundation for designing a more efficient, comprehensive, and intelligent traffic automation system.

A Deep Reinforcement Learning Approach to Traffic Signal Control With Temporal Traffic Pattern Mining Ma, Zhou, Song, and Dai (2021) propose an adaptive traffic signal control system utilizing a CNN–LSTM–A2C deep reinforcement learning architecture [1]. The system processes video feeds and queue statistics, using the CNN and LSTM to mine temporal traffic patterns and feed the state to the A2C agent, demonstrating improved throughput and reduced queue lengths over baseline methods. However, the model struggles under heavy congestion, where queue lengths exceed its learned threshold, and its reliance on pre-mined patterns makes it fragile against traffic anomalies such as accidents.

Mohamad Alizadeh, Shabestary, and Abdulhai (2022) present a framework employing Deep Q-Learning (DQN) to optimize traffic signal phases [2]. The RL agent minimizes overall delay and congestion by learning adaptive policies from raw inputs like vehicle counts and queue lengths. The primary drawbacks include instability under non-recurrent traffic events and an tendency to overfit to specific patterns, limiting generalization across networks. Furthermore, the DQN agent inherently lacks sequential awareness of temporal traffic dynamics.

Learning Ubaid et al. (2022) introduce an intelligent signal automation system using YOLO for real-time object detection to count vehicles, while a rule-based algorithm adjusts green light durations based on density [3]. The system also includes a basic YOLO-based mechanism for emergency vehicle overrides. This computer vision approach, however, suffers from poor performance due to occlusion in dense traffic. The reliance solely on visual recognition makes emergency detection unreliable, and frame-by-frame decisions result in

unstable signal switching without temporal learning.

Multi-Agent Transfer Reinforcement Learning for Adaptive Traffic Signal Control Ge et al. (2022) propose a system using Multi-Agent Reinforcement Learning (MARL) with DQN, where a transfer learning mechanism allows intersections (agents) to share knowledge for better coordination [4]. The framework aims to accelerate training and improve multi-intersection control. The main limitations are the high communication and training complexity involved in MARL, which becomes harder to manage as the number of intersections increases. Learned policies also exhibit weak generalization when applied to different network topologies.

Intelligent Traffic Violation Detection Ravish, Rangaswamy, and Char (2021) present a system using YOLOv3 with Darknet-53 to detect a limited set of violations, specifically helmet absence, seatbelt absence, and red-light violations [5]. The system is capable of high-speed detection (60 fps) and integration with an E-Challan system. Its key weaknesses include being limited to three violations and requiring a manual Region of Interest (ROI) setup. Moreover, accuracy is sensitive to lighting and camera angle, and it lacks license plate recognition.

A Framework for Automatic Detection of Traffic Violations Malik et al. (2023) outline a framework using YOLOv3 to process video feeds via OpenCV and TensorFlow, focusing on flagging red-light violations and estimating speed [6]. The system provides real-time monitoring but is severely limited to only red-light violations and supports only one video feed at a time. Like other similar systems, it requires manual ROI setup, has high computational needs, and its accuracy drops with poor ROI visibility.

Traffic Rules Violation Detection using Machine Learning Techniques Malik et al. (2021) utilize computer vision with R-CNN and SVM features to detect red-signal violations from CCTV footage [7]. Pre-processing techniques are used to reduce noise, and violations are flagged when a vehicle crosses a predefined line during a red signal. The system is strictly limited to signal violations, offers no detection for issues like overspeeding or helmet use, and requires a manual line setup. The processing is also criticized for being slow and lacking multi-stream support.

Smart Traffic Violation Detection System using Artificial Intelligence Kavitha et al. (2023) present a system that pre-processes CCTV footage to reduce noise and employs an R-CNN model to classify vehicles and detect stop-line crossing during a red signal [8]. The use of R-CNN results in a high computational cost, making real-time deployment challenging. Performance is negatively affected by poor lighting and weather conditions, and there is a risk of false positives in heavy traffic dueots to occlusion or camera placement.

Emergency Response Optimization System with Dynamic Traffic Signal Control: (Life Saver) Rajeshwari et al. (2024) introduce the "Life Saver" system, which uses an SOS interface to dispatch ambulances and an ML-based interface for hospital recommendations [9]. The system routes ambulances via real-time APIs and prioritizes their movement by temporarily turning traffic signals green. The major issues involve infrastructure integration, ensuring real-time routing in dynamic conditions, and the limited scalability without cloud

and IoT connectivity for the signals.

Dynamic Traffic Signal Optimization Congestion Management and Emergency Vehicle Prioritization Deoghare et al. (2025) propose a framework using YOLOv8 on real-time images sent to the cloud to detect traffic density and emergency vehicles [10]. The methodology captures real-time traffic images at intersections, transmits them to the cloud, and utilizes YOLOv8 for detecting traffic density and emergency vehicles. Signal timings are dynamically adjusted, and coordination between neighboring signals is achieved using NRF24L01 modules to ensure a smooth flow. Key limitations include limited scalability for large-scale traffic networks and a heavy reliance on continuous internet connectivity for cloud processing and coordination, making the system vulnerable to network disruptions.

Traffic Optimization and Automated Emergency Vehicle Response System Rajesh et al. (2025) detail a system that combines YOLO for vehicle detection with a Reinforcement Learning agent to dynamically adjust signals and prioritize emergency vehicles [11]. The RL agent minimizes waiting times based on vehicle counts and traffic state. Challenges include high computational costs and limited scalability. The system may also exhibit overfitting from simulation-based training and perform less effectively in light traffic.

Smart traffic control system with ambulance detection using image processing Nalini et al. (2024) describe a demand-activated traffic light system controlled by a Siemens PLC, where ambulances are detected using a YOLOv3-CNN model [12]. The signals are controlled to grant a green light to the ambulance's path. The drawbacks are the high initial implementation costs, heavy reliance on technology, and the complex logic required to manage the system across a large urban area.

Traffic Signal Optimization using Real-Time Pedestrian and Vehicle Counts Gomathi et al. (2025) propose optimizing traffic signals using real-time counts of pedestrians and vehicles, detected by TensorFlow's SSD MobileNet V2 with OpenCV [13]. A fallback mechanism is included for feed failures. The system is computationally intensive, requiring GPU acceleration. Accuracy drops under poor lighting or heavy occlusion, and the system lacks emergency vehicle prioritization.

3D-LiDAR-based Pedestrian Detection for Demand-Oriented Traffic Light Control Sprute et al. (2022) present a system using 3D-LiDAR sensors and deep learning (ResNet-50) to detect pedestrians and classify their vulnerabilities for demand-oriented traffic light control [14]. This automates signal requests and reduces waiting times. The approach is costly due to the LiDAR sensors, limited to small intersections, and is less accurate for thin objects. Performance is also poor in bad weather.

Advanced Pedestrian Detection Model with Feature Fusion and Imbalance Resolution Karthika et al. (2025) propose a deep learning model for pedestrian detection that uses feature fusion (color, texture, shape) and techniques to resolve class imbalance [15]. This approach aims for better detection of small and occluded pedestrians. The model, however, has a high computational cost, making real-time implementation challenging, and shows poor performance in low-light or cluttered environments.

Sr. No	Title	Author(s)	Year	Methodology	Drawback
1	*A Deep Reinforcement Learning Approach to Traffic Signal Control With Temporal Traffic Pattern Mining*	Dongfang Ma, Bin Zhou, Xiang Song, Hanwen Dai	2021	The system uses video feeds converted into grid images and queue statistics as input. A combination of CNN and LSTM processes this data, which is then fed into an A2C reinforcement learning agent to decide whether to keep or change the traffic phase. This approach reduces queue lengths and delays while improving throughput compared to fixed-time and baseline RL methods.	The CNN–LSTM–A2C architecture struggles under heavy congestion, where queue lengths exceed the model's learned thresholds, leading to minimal performance gains. Its reliance on pre-mined temporal patterns makes it fragile against anomalies such as accidents or sudden blockages, as the RL agent cannot adapt to unseen traffic dynamics.[1]
2	*A Framework for Adaptive Traffic Signal Control with Deep Reinforcement Learning*	Soheil Mohamad Alizadeh, Shabestary, Baher Abdulhai	2022	The system applies Deep Q-Learning (DQN) to optimize traffic signal phases using raw traffic state inputs such as vehicle counts and queue lengths. The RL agent learns adaptive control policies through simulations, aiming to minimize overall delay and congestion.	The model exhibits instability under non-recurrent traffic events It suffers from overfitting to specific traffic patterns, limiting cross-network generalization. Decisions lack sequential awareness, as the DQN agent does not leverage temporal traffic dynamics. [2]
3	*Intelligent Traffic Signal Automation Based on Computer Vision Techniques Using Deep Learning*	Muhammad Talha Ubaid, Tanzila Saba, Hafiz Umer Draz, Amjad Rehman, Muhammad Usman Ghani Khan,Hoshang Kolivand	2022	The system uses YOLO for real-time object detection on live traffic video feeds to identify and count vehicles at intersections. OpenCV is employed for video frame processing, while a rule-based algorithm adjusts green light durations according to vehicle density. A simple YOLO-based recognition mechanism is also applied to detect emergency vehicles and provide signal overrides.	The system struggles with occlusion and dense traffic. Emergency detection is unreliable, since it relies solely on visual recognition and fails when vehicles are partially blocked. Signal decisions are made frame-by-frame without temporal learning, causing unstable switching under fluctuating traffic conditions.[3]
4	*Multi-Agent Transfer Reinforcement Learning for Adaptive Traffic Signal Control*	Hongwei Ge, Dongwan Gao, Liang Sun, Yaqing Hou	2022	The framework uses Multi-Agent Reinforcement Learning (MARL) with Deep Q-Networks (DQN), where each intersection acts as an agent. A transfer learning mechanism allows knowledge sharing between agents to accelerate training and improve coordination across multiple intersections.	The approach incurs high communication and training complexity, making coordination difficult. Learned policies have weak generalization when applied to different networks.DQN-based MARL grows complex as intersections increase, leading to slower convergence and unstable policies.[4]
5	*Intelligent Traffic Violation Detection*	Roopa Ravish, Shanta Rangaswamy, Kausthub Char	2021	the system uses YOLOv3 with Darknet-53 to detect traffic violations (helmet absence, seatbelt absence, red-light violations) in pre-processed video frames at 60 fps. Detected violations are integrated with an E-Challan system for automated fines.	Limited to three violations, requires manual ROI setup, accuracy depends on lighting and camera angle, needs high-end GPUs for real-time performance, and lacks license plate recognition.[5]

2

Table 2.1: Comparative Analysis of Literature Survey

Sr. No	Title	Author(s)	Year	Methodology	Drawback
6	*A Framework for Automatic Detection of Traffic Violations*	Aman Malik, Suraksha Bisht, Yukta Tripathi, Prashant Kumar, D.R. Gangodkar, Nisha Chandran S.	2023	The system uses YOLOv3 with Darknet-53 to detect vehicles in surveillance videos, processing frames via OpenCV and TensorFlow. It flags red-light violations, estimates speed, and enables real-time monitoring through a GUI, with manual ROI setup for infrastructure detection.	The system is limited to red-light violations, lacks number plate recognition and other offense detection, and supports only one video feed at a time. It requires manual ROI setup, has high computational needs, and accuracy decreases with poor ROI visibility or varied signals.[6]
7	*Traffic Rules Violation Detection using Machine Learning Techniques*	Aman Malik, Suraksha Bisht, Yukta Tripathi, Prashant Kumar, D.R. Gangodkar, Nisha Chandran S.	2021	The system applies computer vision and deep learning to detect red-signal violations from CCTV footage. Pre-processing (grayscale, Gaussian blur, background subtraction, thresholding, contour detection) reduces noise, while R-CNN with CNN features and SVM classifies vehicles Violations are flagged when vehicles cross predefined lines during a red signal.	Limited to signal violations, no detection for over speeding/helmet/lane issues. Needs manual line setup and clear footage; accuracy drops in poor lighting or occlusion. Processing is slow, lacks multi-stream support, and license plate recognition is basic.[7]
8	*Smart Traffic Violation Detection System using Artificial Intelligence*	Kavitha A, Kanimozhi T, Dharani J, Kishore S, Praveen C, Vignesh K	2023	CCTV footage is pre-processed using grayscale conversion, Gaussian blurring, background subtraction, binary thresholding, and contour detection to reduce noise. Vehicle detection is carried out with an R-CNN model to classify vehicles. Violations are detected when a vehicle crosses the stop line during a red signal.	The model has high computational cost due to R-CNN, making real-time deployment difficult. Performance reduces in poor lighting and weather conditions. False positives may occur due to occlusion, camera placement, or heavy traffic scenarios.[8]
9	*Emergency Response Optimization System with Dynamic Traffic Signal Control: (Life Saver)*	Dr. Rajeshwari B.S, Vinay Y, Venu Gopal V, Tauksik Anil Kumar, Viraj Chowdarapu	2024	The system uses an SOS interface to track users' locations and dispatch the nearest ambulance, while an ML-based health interface recommends nearby hospitals. Ambulances are routed efficiently using real-time APIs, and traffic signals are temporarily turned green near approaching ambulances to prioritize their movement.	Drawbacks include infrastructure integration, real-time routing under dynamic conditions, environmental and technical constraints, and limited scalability without cloud and IoT-based signal connectivity.[9]
10	*Dynamic Traffic Signal Optimization Congestion Management and Emergency Vehicle Prioritization*	Seema U. Deoghare, Yash Dhond, Saakshi Gaikwad, Parth Gorkhe	2025	The system captures real-time traffic images at intersections and sends them to the cloud, where YOLOv8 detects traffic density and emergency vehicles. Signal timings are dynamically adjusted, and neighboring signals coordinate via NRF24L01 modules to ensure smooth traffic flow.	The system has limited scalability for large traffic networks and relies on continuous internet connectivity, making it vulnerable to network disruptions.[10]

Sr. No	Title	Author(s)	Year	Methodology	Drawback
11	*Traffic Optimization and Automated Emergency Vehicle Response System*	Dr. R. Rajesh, Kathir MN, Hariprasana Senthilkumar, Parthiban SK	2025	The system uses YOLO for real-time vehicle detection and a Reinforcement Learning agent to dynamically adjust traffic signals. The RL agent decides signal changes based on vehicle counts, traffic state, and a reward function that minimizes waiting times and prioritizes emergency vehicles, with dynamic adjustments ensuring fairness and emergency overrides.	The system faces high computational costs and limited scalability, with potential overfitting from simulation-based training. It also encounters challenges in integrating multiple sensors and may be less effective in light traffic or when non-vehicle interference occurs.[11]
12	*Smart traffic control system with ambulance detection using image processing*	Dr.M.Nalini, S.Sandhya, S.Shiwan S.Charumathi, G.Maharajothi	2024	The system uses demand-activated traffic lights controlled by a Siemens PLC, with ambulances detected via a YOLOv3-CNN model. Signals turn green for the ambulance and red for all others, resuming normal cycles once it passes.	The system has high initial implementation costs, depends heavily on technology, and involves complex logic that may be challenging to manage across large urban networks.[12]
13	*Traffic Signal Optimization using Real-Time Pedestrian and Vehicle Counts*	Gomathi S, Thiruvenkadakrishnan S, Sharan Sampath Kumar V, B Rithish Kumar G	2025	The system uses real-time video and computer vision to dynamically optimize traffic signals. TensorFlow's SSD MobileNet V2 with OpenCV detects pedestrians and vehicles, while modules for counting and signal optimization adjust green lights based on traffic. A fallback ensures operation during feed failures, with performance measured by wait times, traffic flow, safety, and environmental impact.	The approach is computationally intensive, requiring GPU acceleration for real-time processing. Its accuracy may drop under poor lighting, adverse weather, or heavy traffic occlusion. Scalability remains challenging for large networks, and the system lacks integration of emergency vehicle prioritization.[13]
14	*3D-LiDAR-based Pedestrian Detection for Demand-Oriented Traffic Light Control*	Dennis Sprute, Florian Hufen, Tim Westerhold, Holger Flatt	2022	Uses 3D-LiDAR sensors and deep learning (ResNet-50) to detect pedestrians and classify vulnerabilities (e.g., elderly, stroller, cyclist). Automates signal requests, reduces waiting time, and adjusts green phase based on needs.By leveraging this detailed real-time information, the system automates signal requests, dynamically reduces pedestrian waiting times, and intelligently adjusts the green phase duration based on the needs of the detected users.	The approach is costly due to 3D-LiDAR sensors, limited to small intersections, and less accurate for thin objects. Performance drops in bad weather, and reliance on a custom dataset makes large-scale deployment difficult.[14]
15	*Advanced Pedestrian Detection Model with Feature Fusion and Imbalance Resolution*	R. Karthika, R. Kavitha, K. Nandhini, S. Elakkiya, T. Deepa	2025	Combines color, texture, and shape features through fusion and addresses class imbalance using adaptive sampling/weighted loss. Uses deep learning with multi-scale feature maps for better detection of small and occluded pedestrians.	High computational cost, poor performance in low-light or cluttered environments, and challenging real-time implementation due to complexity[15]

Chapter 3

Project Design

The project design of the Intelligent Traffic Signal Control system aims to create a well-structured and functional architecture that translates conceptual ideas into a practical and deployable framework. This phase defines the development of core modules, their interactions, and the flow of data that enables real-time adaptability in complex traffic environments. The design emphasizes modularity, scalability, and robustness, ensuring that the system can handle both routine traffic conditions and critical emergency scenarios while maintaining transparency and safety.

The project design integrates multiple intelligent components into a cohesive structure. The Perception Agent interprets traffic conditions using computer vision, the PPO Traffic Agent applies reinforcement learning to optimize signal timings dynamically, and the Meta Controller ensures safety-prioritized overrides for emergency vehicles. The Planning Agent consolidates recommendations into executable signal strategies. Collectively, these modules form an adaptive ecosystem capable of balancing traffic flow efficiency, emergency handling, pedestrian safety, and accountability.

Additionally, the project design ensures compatibility with both simulation environments such as SUMO and real-world smart city infrastructures, enabling smooth transition from testing to deployment. The following sections present the architectural elements, workflow strategies, and integration logic that form the backbone of this AI-powered traffic management project.

3.1 Existing System

The challenge of traffic congestion has long been addressed through a variety of traffic management systems, ranging from fixed-timer signal controllers to more advanced sensor-driven and AI-based approaches. While these systems have contributed significantly to easing traffic in many contexts, they exhibit limitations when applied to highly dynamic and heterogeneous traffic environments such as those found in Indian cities. A review of existing systems provides useful insights into the strengths and weaknesses of prior work and sets the foundation for the proposed AI-driven solution.

- **Fixed-Time Signal Control Systems:** Traditional traffic signal control methods rely on fixed-time scheduling, where green, yellow, and red light durations are pre-

determined based on historical data. These systems are simple to implement and cost-effective, making them widely used in many urban areas. However, they are static in nature and do not adapt to real-time fluctuations in traffic. This often results in inefficient allocation of green time—lanes with heavy congestion remain blocked while empty lanes still receive unnecessary clearance. In rapidly urbanizing contexts, such as India, fixed-time systems are increasingly ineffective due to unpredictable traffic patterns, absence of strict lane discipline, and high variability in vehicle flow.

- **Actuated and Sensor-Based Systems:** To overcome the rigidity of fixed-time controllers, actuated systems use inductive loop detectors, infrared sensors, or camera-based counters to estimate real-time vehicle density. These adaptive systems adjust the duration of signal phases depending on traffic demand. For example, if vehicle queues are detected in one direction, the controller can extend the green phase until the queue clears. While these systems represent an improvement, they still rely on rule-based decision-making and limited data inputs. They often fail in cases of complex junctions, irregular traffic behavior, or unexpected events such as emergency vehicles and accidents. Furthermore, installing and maintaining physical sensors is costly and prone to technical failures, limiting scalability.

- **Intelligent Transportation Systems (ITS):** Intelligent Transportation Systems (ITS) mark the beginning of integrating artificial intelligence into traffic management. These systems combine real-time data collection, communication technologies, and decision-support algorithms to optimize traffic flow. For instance, centralized ITS platforms may aggregate live traffic feeds from multiple intersections and attempt to coordinate signals across a network. However, ITS implementations are often data-heavy, infrastructure-intensive, and dependent on advanced connectivity. While effective in developed countries, their direct replication in Indian conditions faces challenges such as poor road infrastructure, heterogeneous traffic, and limited budget allocations.

- **Reinforcement Learning-Based Traffic Control:** Recent research has applied reinforcement learning (RL) to traffic management, treating the signal controller as an agent that learns optimal actions based on states (traffic density, waiting time, queue length) and rewards (reduced congestion or waiting time). Algorithms such as Q-learning and Deep Q-Networks (DQN) have shown promise in simulations, achieving better adaptability than fixed or rule-based methods. However, these approaches often optimize solely for throughput and do not account for safety-critical events, such as emergency vehicle prioritization or pedestrian crossings. Moreover, RL systems are frequently criticized as black boxes—while they may produce effective results, their decision-making process lacks transparency, making it difficult for traffic authorities to trust and deploy them in real-world scenarios.

- **Computer Vision and Deep Learning Approaches:** With the rise of affordable cameras and advances in computer vision, several systems now use YOLO, CNNs, and other object detection techniques to monitor traffic density. These methods can count vehicles, detect congestion, and even identify violations such as red-light jumping. However, such vision-based systems are usually used in isolation for monitoring purposes rather than as part of an integrated adaptive signal control framework. They also face challenges in poor visibility conditions such as heavy rain, fog, or nighttime traffic, which are common in real-world scenarios.

- **Hybrid Systems Combining AI and Rule-Based Logic:** Some of the more advanced systems integrate reinforcement learning with rule-based controllers to balance adaptability with safety. For example, an RL agent may determine signal timings under normal conditions, but rule-based logic takes over during emergency scenarios. These hybrid approaches offer better safety assurances but often lack explainability—while they may override certain decisions, they do not provide interpretable reasoning behind the change. As such, they remain limited in terms of transparency and authority acceptance.

3.2 Proposed System

The proposed intelligent traffic management system aims to create an adaptive and safe approach to controlling traffic flow at busy intersections using a combination of reinforcement learning, computer vision, and large language model–based reasoning. As illustrated in Fig. 3.1, the architecture begins with real-time video input captured from multiple directions at the junction. This data is processed to interpret vehicle density, identify pedestrians, and detect the presence of emergency vehicles. The first stage involves an LLM-based agent for scene understanding, which transforms raw video input into a structured representation of traffic conditions. This agent ensures that the system does not simply count vehicles but contextualizes the situation to reflect real-world conditions, which is particularly relevant in complex urban environments.

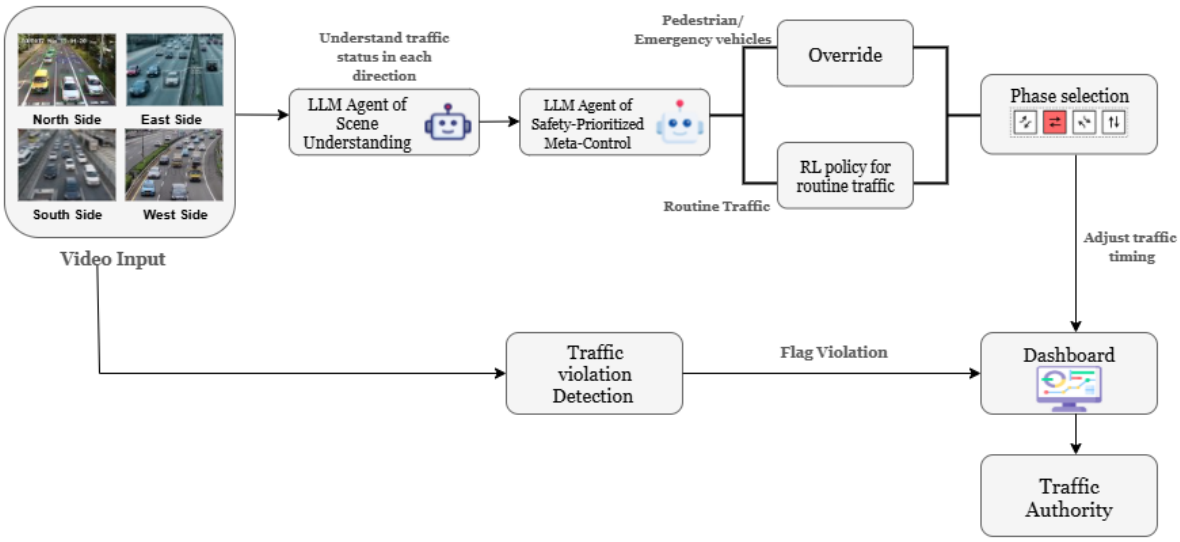

Figure 3.1: System Architecture of the Proposed Intelligent Traffic Management System

Once the scene is understood, the information flows to the safety-prioritized meta controller. This module plays a central role in balancing efficiency and safety. Under routine traffic conditions, a reinforcement learning policy is used to dynamically adjust signal timings, ensuring maximum throughput and minimal waiting times. However, when pedestrians or emergency vehicles are detected, the safety meta-controller overrides the routine policy

and assigns priority to those movements. This layered decision-making ensures that the system remains both adaptive and safe, avoiding the rigidity of conventional fixed-timing methods.

The decision-making process further incorporates a collaborative reasoning module, which integrates inputs from both the reinforcement learning policy and the safety controller to finalize the appropriate traffic signal phase. The phase selection unit then determines which direction receives the green signal and for how long, based on the combined logic of efficiency and priority. This real-time adaptability helps reduce congestion, improves travel time, and accommodates critical scenarios without human intervention.

Parallel to this flow, the system also integrates traffic violation detection. The same video input is analyzed using computer vision techniques to identify violations such as red-light jumping, lane indiscipline, or stop-line crossings. These violations are flagged and reported to the dashboard, creating a transparent enforcement mechanism.

The dashboard serves as an interface for traffic authorities, providing live visualizations of system decisions, detected violations, and overall traffic conditions. This human-in-the-loop design ensures that the system remains explainable and trustworthy, while also offering authorities the option to intervene when needed. By combining adaptive learning, safety-aware reasoning, and automated enforcement, the proposed system offers a practical step forward in smart traffic control.

3.2.1 Critical Components of System Architecture

A. Perception and Detection Module

The perception module serves as the sensory foundation of the system, capturing live visual data from surveillance cameras positioned at intersections. Using YOLO-based object detection, the system identifies vehicles, pedestrians, and emergency vehicles in real time. The detected objects are categorized by type (car, bike, truck, ambulance, pedestrian), and their spatial positions are analyzed to estimate lane density and waiting time. This module also contributes to pedestrian safety by recognizing crosswalk occupancy and generating alerts for the meta-controller. The processed traffic data is structured into a numerical traffic state vector, which is passed to the decision-making component for further evaluation.

B. PPO Decision-Making Agent

The Proximal Policy Optimization (PPO) agent forms the decision core of the architecture. As shown in Fig. 3.2, it continuously interacts with the environment by observing traffic states and outputting optimal actions in terms of phase selection and green light duration. The PPO architecture includes two neural networks: an Actor and a Critic. The Actor determines the next signal phase and its duration based on the learned policy, while the Critic evaluates the chosen action by estimating its value. The agent is trained using Generalized Advantage Estimation (GAE) and a clipped surrogate loss to ensure stable updates. Through continuous interaction, the PPO model learns to minimize total waiting time, reduce queue lengths, and maintain smooth traffic flow.

C. Safety-Prioritized Meta Controller

The Meta Controller supervises the PPO's actions and ensures safety compliance under exceptional conditions. When the perception module detects an emergency vehicle, the meta controller immediately overrides the PPO output, granting right-of-way to the corresponding lane. It also enforces traffic safety protocols such as all-red clearance phases during transitions, preventing potential collisions. This layer acts as a high-level policy regulator that ensures operational robustness in critical or unpredictable traffic scenarios.

D. Planning and Execution Module

Once the optimal phase and duration are selected—either by the PPO agent or through an emergency override—the planning module generates executable control commands for the Traffic Light Controller. These commands are sent to the intersection's signal units through a microcontroller interface or simulated in SUMO for training. The planning module also ensures that phase transitions occur smoothly and that signal durations remain within safety thresholds. Feedback from this stage is used to compute environmental rewards that reinforce learning in the PPO training loop.

E. Traffic Violation Detection System

In parallel, the Violation Detection Subsystem operates using OpenCV-based video analytics. It identifies instances such as red-light running, lane violations, and pedestrian rule breaches. Captured frames are timestamped and logged in a secure database, enabling further review by traffic authorities. This module contributes to long-term enforcement and system transparency.

F. Data Logging and Continuous Learning

All simulation data—state transitions, rewards, and policy outputs—are stored in structured buffers for continuous improvement. This historical data can be leveraged to retrain the PPO model periodically, enabling the system to adapt to evolving traffic patterns and seasonal variations.

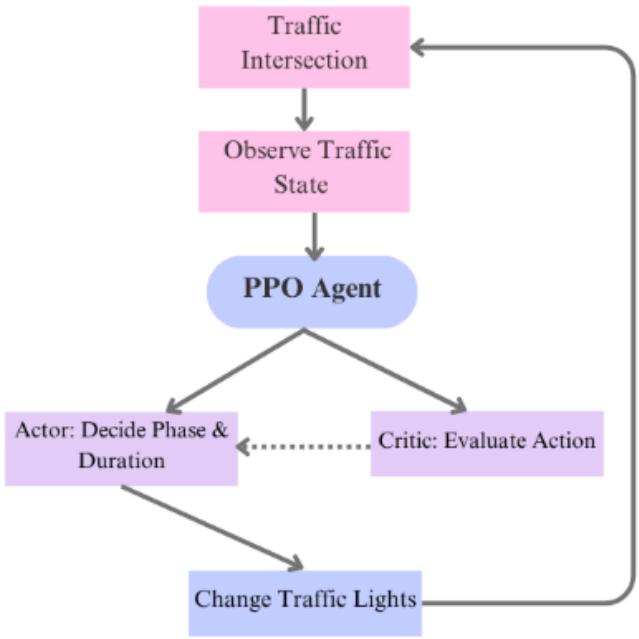

Figure 3.2: Critical Components of System Architecture

In essence, the collaboration of these six critical components enables the system to perceive real-time traffic dynamics, make intelligent policy-based decisions, and enforce safety measures effectively. Together, they form an intelligent, adaptive, and scalable framework for modern urban traffic management.

3.3 System Diagrams

3.3.1 UML Diagram

The comprehensive design and adaptive functionality of the VISTA Intelligent Traffic Management System are precisely communicated through several key visual models. These diagrams, rooted in Data Flow Analysis and the Unified Modeling Language (UML), serve as the definitive technical blueprint, detailing the system's architecture, behavioral logic, and time-sensitive component interactions. They are crucial tools for clarifying the system's operational complexity for developers, stakeholders, and regulatory bodies.

To provide a complete understanding of the system's operation, this section employs three foundational models. The Data Flow Diagram (DFD), which we discussed earlier, visualizes the movement and transformation of traffic data as it progresses through the system's analytical and decision-making stages. The Use Case Diagram identifies all system functionalities, clearly defining the scope of VISTA and the specific roles of external actors, such as the Traffic Authority and Junction Cameras.

Finally, the Sequence Diagram provides a detailed, time-ordered perspective on how the system's core components—particularly the LLM Agents and policy modules—interact to achieve the critical objective of signal adjustment. Together, these models meticulously illustrate the complete lifecycle of data within VISTA, from raw video ingestion to the final deployment of an optimized or safety-prioritized traffic signal timing plan.

3.3.2 DataFlow Diagram

The VISTA system's operation is defined by a structured Data Flow Diagram (DFD), which visually maps how information moves through the system, illustrating processes, data stores, and external entities. As shown in Figure 3.3: VISTA System Data Flow Diagram, the pipeline begins with Raw Video Streams, which are simultaneously analyzed to extract essential traffic metrics: Vehicle Count Per Lane, Emergency Vehicle Count, and Pedestrian Count, along with parallel generation of Traffic Violation Data.

This quantitative data is then synthesized by the LLM Agent of Scene Understanding, which creates a comprehensive Scenario Description. This key step transforms raw numbers into a contextual narrative (e.g., "North side congested, Emergency vehicle approaching East") that enables cognitive decision-making. The description feeds into the LLM Agent of Safety-Prioritized Meta-Control.

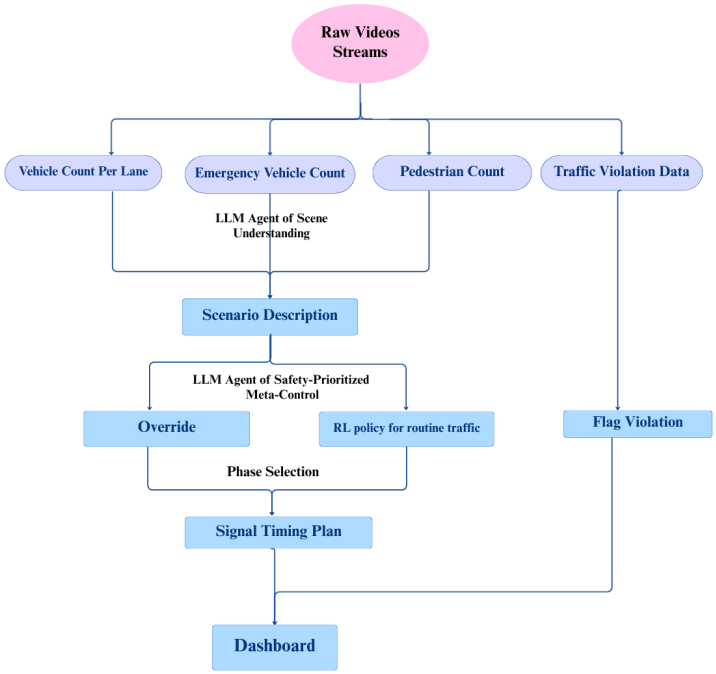

Figure 3.3: VISTA System Data Flow Diagram

This quantitative data is then synthesized by the LLM Agent of Scene Understanding, which creates a comprehensive Scenario Description. This key step transforms raw numbers into a contextual narrative (e.g., "North side congested, Emergency vehicle approaching East") that enables cognitive decision-making. The description feeds into the LLM Agent of Safety-Prioritized Meta-Control.

The Meta-Control agent dictates the strategy: it activates the RL Policy for Routine Traffic for efficiency, or initiates an Override for immediate safety in the presence of emergency vehicles or high pedestrian counts. The resulting policy outputs are used by Phase Selection to compute the final Signal Timing Plan. All outputs, including the Signal Timing Plan and Flag Violation data, are consolidated on the Dashboard for immediate deployment to the junction signals and for reporting to the Traffic Authority. This pipeline ensures responsive, safe, and efficient traffic management.

3.3.3 Use Case Diagram

The Use Case Diagram in Figure 3.4 illustrates the interaction between different actors and the proposed AI-Powered Smart Traffic Automation System. It highlights the system's core functionalities while showing how external entities such as drivers, traffic authorities, and junction infrastructure connect with its modules. The diagram demonstrates the services provided by the system, including traffic violation detection, adaptive phase control, and emergency handling.

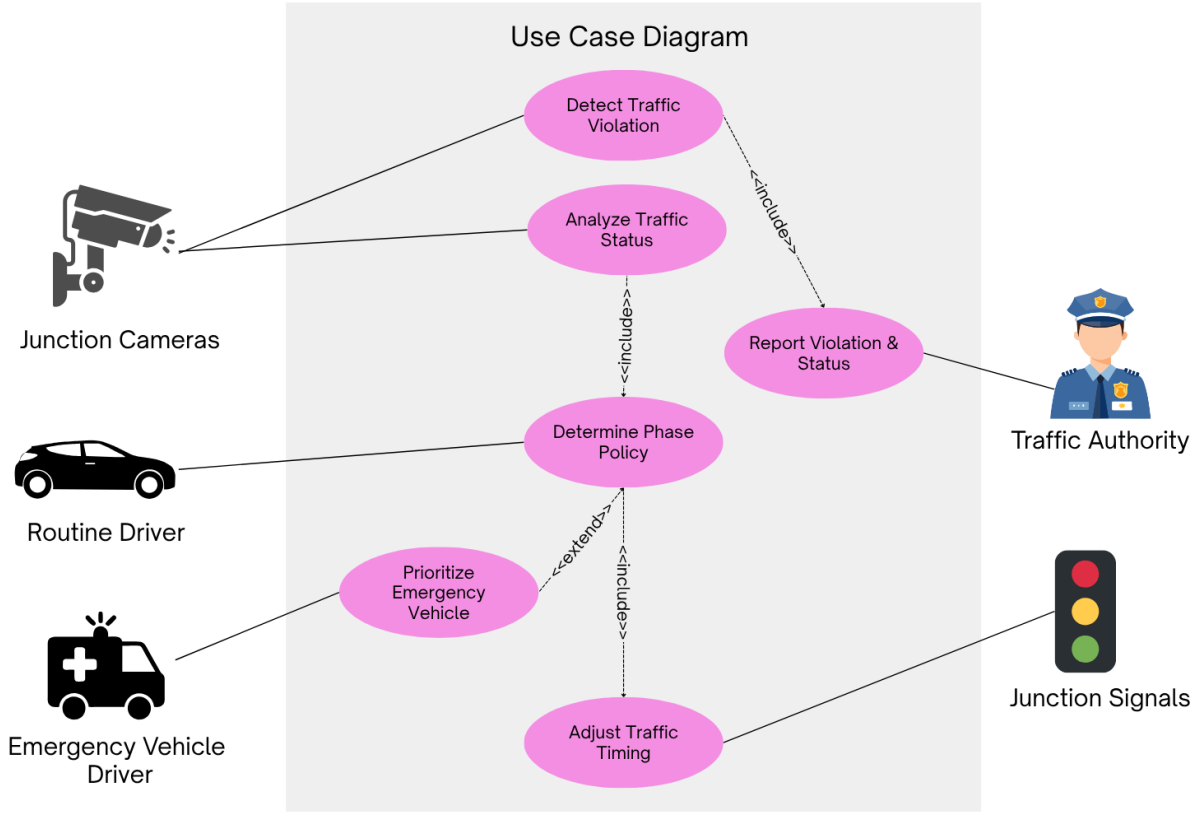

Figure 3.4: VISTA System Use Case Diagram

Junction Cameras serve as the primary sensory inputs, monitoring vehicle movement and road behavior. They enable key use cases such as Detect Traffic Violation and Analyze Traffic Status, generating data that informs higher-level decisions. Routine Drivers indirectly engage with the system through the Determine Phase Policy use case, which dynamically adjusts signal timings to reduce waiting times and congestion.

In special scenarios, Emergency Vehicle Drivers benefit from the Prioritize Emergency Vehicle use case, where the system overrides or extends phases to ensure quick passage. This feature reinforces the system's safety-first design, allowing ambulances or fire trucks to respond faster in critical situations.

The system also interacts with the Traffic Authority through the Report Violation and Status use case. This ensures that red-light jumping, lane misuse, and other violations are logged and reported automatically, reducing manual monitoring and improving enforcement accuracy.

At the operational level, the Determine Phase Policy is the central decision process. Based on traffic analysis and emergency inputs, it selects the optimal phase and activates Adjust Traffic Timing. These decisions are then executed by Junction Signals, which translate policies into real-world traffic control.

Overall, the diagram depicts how detection, decision-making, and enforcement integrate within a single system. It addresses both routine traffic optimization and emergency responsiveness while promoting accountability through automated reporting. The primary objective of this diagram is to represent the functional scope of the system and emphasize its ability to meet user goals—whether reducing congestion, clearing emergency routes, or supporting traffic authorities. By capturing these use cases, the system is shown as a scalable and intelligent solution for modern urban intersections.

3.3.4 Sequence Diagram

The operation of the VISTA Intelligent Traffic Management System is illustrated through a Figure 3.5: VISTA System Sequence Diagram, which serves as a specialized behavioral UML model detailing the time-ordered interactions and message exchanges between the system's components (Lifelines). This diagram clearly captures the flow of control and data, demonstrating the system's rapid response mechanism, particularly the decision-making divergence between routine optimization and safety prioritization.

The sequence begins with the :Junction Cameras providing the streamVideo(4 directions) input to the :Video Processing module. This module executes detectCounts() and forwards the extracted traffic metrics—including the critical Emergency=True flag if detected—to the :LLM Scene Agent. The Scene Agent's primary function is to achieve situational awareness by executing synthesizeDescription(), transforming raw data into a narrative sendScenarioDescription() for the meta-controller.

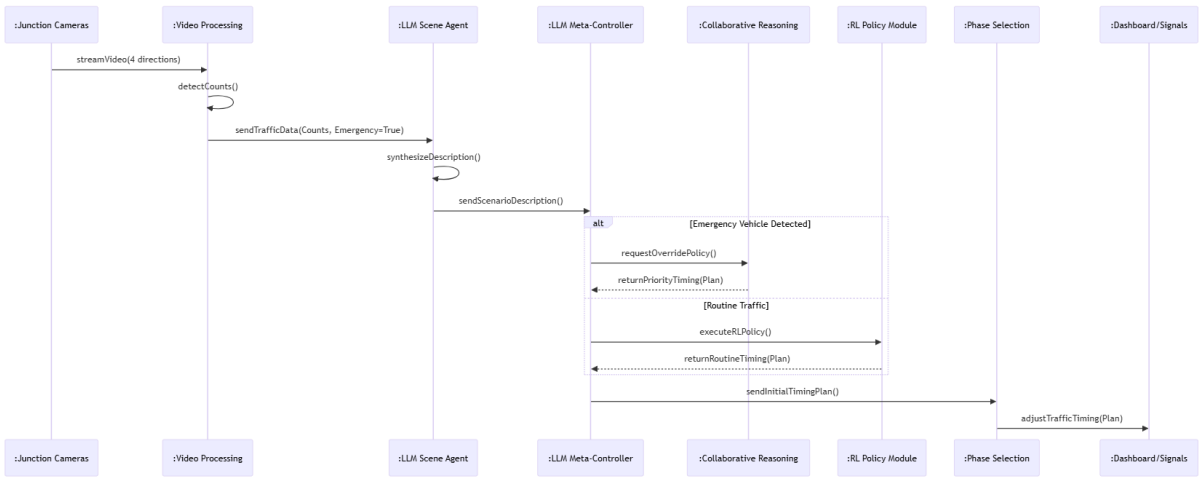

Figure 3.5: VISTA System Sequence Diagram

The core of the system's adaptability resides in the :LLM Meta-Controller, which uses an alt (alternative) fragment to govern the subsequent policy. If the description indicates [Emergency Vehicle Detected], the Meta-Controller immediately calls the :Collaborative Reasoning module via requestOverridePolicy(), which returns a safety-prioritized returnPriorityTiming(Plan). Conversely, for [Routine Traffic], the Meta-Controller delegates control

to the :RL Policy Module via executeRLPolicy() to retrieve an optimized returnRoutineTiming(Plan). This dynamic arbitration ensures that safety protocols supersede routine efficiency when necessary.

Following the policy determination, the :LLM Meta-Controller sends the resulting plan via sendInitialTimingPlan() to the :Phase Selection module. The Phase Selection module is responsible for final validation and deployment, concluding the sequence by dispatching the command adjustTrafficTiming(Plan) to the :Dashboard/Signals lifeline. This time-sensitive interaction confirms the system's capability to execute a rapid and context-aware control loop, moving from video capture to signal adjustment in a defined, auditable sequence.

Chapter 4

Project Implementation

This chapter provides a comprehensive overview of the implementation phase of the project, detailing the technical execution and development process. It includes key code snippets thatillustrate core functionalities, step-by-step instructions for accessing and interacting with the system, and a timeline that outlines the progression of development milestones. This chapteraims to connect the conceptual framework with its practicalexecution, illustrating how the proposed design was methodically developed into a working system.

4.1 Code Snippets

This section showcases key code excerpts from the smart traffic control system, highlighting core features such as PPO-based traffic signal optimization, LLM-powered traffic state descriptions, and emergency vehicle detection with priority handling. Each snippet is briefly explained with a diagram to illustrate its workflow, demonstrating how machine learning, reinforcement learning, and computer vision work together for efficient traffic management and emergency response.

```python
# PPO Policy Update (Clipped) — ppo_agent.py
for _ in range(train_pi_iters):
    with tf.GradientTape() as tape:
        # Actor predicts phase (categorical) and duration (continuous) actions
        phase_logits, duration_mu, duration_logstd = self.actor([traffic_state_buf, phase_index_buf])

        # PPO clipped loss for phase (discrete action)
        phase_probs = tf.nn.softmax(phase_logits)
        logp_phase = tf.math.log(tf.reduce_sum(phase_probs * tf.one_hot(phase_buf, self.phase_dim), axis=1) + 1e-8)
        ratio_phase = tf.exp(logp_phase - logp_phase_buf)
        min_adv_phase = tf.where(adv_buf > 0, (1 + self.clip_ratio) * adv_buf, (1 - self.clip_ratio) * adv_buf)
        phase_loss = -tf.reduce_mean(tf.minimum(ratio_phase * adv_buf, min_adv_phase))

        # PPO clipped loss for duration (continuous action)
        duration_std = tf.exp(duration_logstd)
        logp_duration = -0.5 * ((duration_buf - duration_mu)/(duration_std + 1e-8))**2 \
                        - tf.math.log(duration_std + 1e-8) - 0.5 * np.log(2*np.pi)
        ratio_duration = tf.exp(logp_duration - logp_duration_buf)
        min_adv_duration = tf.where(adv_buf > 0, (1 + self.clip_ratio) * adv_buf, (1 - self.clip_ratio) * adv_buf)
        duration_loss = -tf.reduce_mean(tf.minimum(ratio_duration * adv_buf, min_adv_duration))

        # Combine both losses for final policy update
        pi_loss = phase_loss + duration_loss

    actor_vars = self.actor.trainable_variables + [self.actor.duration_logstd]
    self.pi_optimizer.apply_gradients(zip(tape.gradient(pi_loss, actor_vars), actor_vars))
```

Figure 4.1: PPO Policy Update Function

The Proximal Policy Optimization (PPO) algorithm is employed to train the reinforcement learning agent that controls the traffic signal timing. Figure 4.1 illustrates the policy update process of the PPO agent, where both the *phase* (which traffic signal to activate) and the *duration* (for how long it remains active) are optimized simultaneously.

The presented code implements the **clipped policy gradient update**, a core feature of PPO that ensures stable and controlled learning.

- The **actor network** outputs probabilities for each signal phase (discrete action) and a Gaussian distribution for the green light duration (continuous action).
- The algorithm computes *ratios* between new and old policy probabilities to determine the policy adjustment magnitude.
- The **clipped objective function** restricts the policy update step to remain within a safe range, improving convergence stability.
- The combined losses for both action components are used to update the actor network parameters through gradient descent.

Overall, this PPO-based optimization allows the agent to learn adaptive and efficient traffic signal strategies that minimize waiting time and improve overall traffic flow.

```python
# Cached description update — llm_description_service.py
def update_descriptions(self, traffic_data):
    current_time = time.time()
    with self.cache_lock:
        # Use cached text if it's still fresh and fully loaded
        fresh = current_time - self.description_cache['last_update'] < self.description_cache['cache_duration']
        ready = all(d != "Loading..." for d in self.description_cache['camera_descriptions'])
        if fresh and ready:
            return self.description_cache

        # Generate new camera-level descriptions and overall summary
        cams = [self.generate_camera_description(i, traffic_data) for i in range(4)]
        overall = self.generate_overall_summary(traffic_data)

        # Update cache with new data and timestamp
        self.description_cache.update({
            'camera_descriptions': cams,
            'overall_summary': overall,
            'last_update': current_time
        })
        return self.description_cache
    def generate_camera_description(self, camera_index, traffic_data):
```

Figure 4.2: LLM agent Description Service

The LLM-based description service is designed to generate textual summaries of the traffic state in real-time. Figure 4.2 illustrates the update mechanism, which ensures that the descriptions remain current while efficiently utilizing cached information.

The function checks whether the cached descriptions are still valid based on a predefined cache duration and whether all camera descriptions are fully loaded. If the cache is fresh and ready, it is returned directly, minimizing redundant computations and improving performance.

If the cache is outdated, the function regenerates:

- Individual camera-level descriptions using the `generate_camera_description` method.
- An overall traffic summary using the `generate_overall_summary` method.

The cache is then updated with the new descriptions and timestamp, ensuring that subsequent requests receive the latest information.

This mechanism allows the system to provide timely, accurate textual insights of the traffic conditions without overloading computational resources.

```python
# Emergency duration and served directions — meta_controller.py
def _get_phase_directions(self, phase_idx):
    # Maps each traffic light phase to the directions it serves
    phase_mappings = {
        0: ['North', 'South'],   # Through N-S
        1: ['North', 'South'],   # Left turns N-S
        2: ['East', 'West'],     # Through E-W
        3: ['East', 'West'],     # Left turns E-W
    }
    return phase_mappings.get(phase_idx, [])

def _calculate_emergency_duration(self, severity, emergency_dirs):
    # Base time adjusted by severity (urgency) and number of active approaches
    base = self.min_emergency_duration          # e.g., 45 s
    severity_bonus = int(severity * 20)         # adds time for high severity
    direction_bonus = len(emergency_dirs) * 5   # adds 5 s per active approach

    # Keep duration within defined safety limits
    total = base + severity_bonus + direction_bonus
    return min(max(total, self.min_emergency_duration), self.max_emergency_duration)
def _handle_emergency_phase(self, emergency_info):
```

Figure 4.3: Emergency Handling Module: Direction Mapping and Duration Calculation

The emergency handling module dynamically adjusts traffic signal phases to prioritize the passage of ambulances and other high-priority vehicles. Figure 4.3 illustrates how the system determines which directions are affected and computes the required green signal duration based on emergency severity.

The function `_get_phase_directions` maps each traffic signal phase to the directions it serves, allowing the system to identify which lanes need priority.

The `_calculate_emergency_duration` function computes the green light duration by considering:

- A base minimum emergency duration.
- An additional time proportional to the severity of the emergency.
- A bonus based on the number of active directions requiring priority.

The final duration is clipped within pre-defined minimum and maximum limits to ensure safety and prevent excessive delays to other traffic.

This mechanism ensures that emergency vehicles receive priority passage through intersections, improving response times while maintaining overall traffic flow efficiency.

```python

def detect_vehicle_crash(frame_results, vehicle_classes=None, iou_threshold=0.25):
    """Detect crash candidates by overlapping vehicle boxes in one frame.

    Args:
        frame_results: YOLO frame result object with `.boxes.xyxy` and `.boxes.cls`.
        vehicle_classes: List of class IDs considered vehicles.
        iou_threshold: Overlap threshold to flag a crash.

    Returns:
        crash_events: List of tuples (box_a, box_b, iou)
    """
    if vehicle_classes is None:
        # COCO vehicle classes for many YOLO models: car, motorbike, bus, truck
        vehicle_classes = [2, 3, 5, 7]

    crash_events = []
    if frame_results is None or frame_results.boxes.data is None:
        return crash_events

    boxes = frame_results.boxes.xyxy.cpu().numpy()
    classes = frame_results.boxes.cls.int().cpu().numpy()

    # Keep only vehicle hits
    vehicle_indices = [i for i, cls in enumerate(classes) if int(cls) in vehicle_classes]
    vehicles = [boxes[i] for i in vehicle_indices]

    # Pairwise overlap check
    for i in range(len(vehicles)):
        for j in range(i + 1, len(vehicles)):
            overlap = iou(vehicles[i], vehicles[j])
            if overlap > iou_threshold:
                crash_events.append((vehicles[i].tolist(), vehicles[j].tolist(), float(overlap)))

    return crash_events

```

Figure 4.4: Real-Time Accident Detection and Alert Logic

The Accident Detection Module is implemented as a high-priority safety layer within the VISTA surveillance suite. Figure 4.4 illustrates the backend logic responsible for processing video frames, identifying vehicular collisions, and triggering the automated alert system. The script utilizes the YOLOv11 framework to perform per-frame inference, ensuring that safety incidents are captured with minimal latency.

- Model Inference: The system loads a specialized accident model.pt weight file to detect "Accident" classes within the live four-junction camera feed.
- Confidence Filtering: A strict probability threshold is applied to the detection results to filter out environmental noise and ensure that only high-confidence collisions trigger a system response.
- State Management: The code maintains a global accident detected flag and an accident counter, which increments upon each unique detection to facilitate forensic data logging.

- Visual Alert Trigger: When a collision is confirmed, the logic dynamically updates the dashboard interface with an "ACCIDENT DETECTED!" overlay and logs a precise Last Event Timestamp.
- Surveillance Integration: The module operates in parallel with the traffic control logic, providing authorities with a real-time safety audit without interrupting the RL-based signal optimization.

Overall, this automated detection script replaces the need for constant manual monitoring, allowing for a faster emergency response and improved data accuracy for the Traffic Authority.

```python
def is_red_light_on(frame, red_polygon, red_thresh=0.02):
    """Detect red light by HSV color mask inside the red signal polygon."""
    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
    lower1, upper1 = np.array([0, 50, 50]), np.array([10, 255, 255])
    lower2, upper2 = np.array([170, 50, 50]), np.array([180, 255, 255])
    mask1 = cv2.inRange(hsv, lower1, upper1)
    mask2 = cv2.inRange(hsv, lower2, upper2)
    red_mask = cv2.bitwise_or(mask1, mask2)

    roi_mask = np.zeros(frame.shape[:2], dtype=np.uint8)
    cv2.fillPoly(roi_mask, [red_polygon], 255)
    red_region = cv2.bitwise_and(red_mask, red_mask, mask=roi_mask)

    red_pixels = cv2.countNonZero(red_region)
    total_pixels = cv2.countNonZero(roi_mask)
    return (red_pixels / max(1, total_pixels)) > red_thresh


def detect_signal_jump(boxes_xyxy, classes, class_names, target_labels, roi_polygon, red_light_on):
    """Mark violations if vehicles in ROI while red light is on."""
    violations = []
    if not red_light_on:
        return violations

    for box, cls in zip(boxes_xyxy, classes):
        label = class_names[int(cls)]
        if label not in target_labels:
            continue

        x1, y1, x2, y2 = map(int, box)
        cx, cy = (x1 + x2) // 2, y2
        if cv2.pointPolygonTest(roi_polygon, (cx, cy), False) >= 0:
            violations.append({'label': label, 'box': [x1, y1, x2, y2], 'center': (cx, cy)})

    return violations
```

Figure 4.5: Red Light Violation Detection Logic

The Red Light Violation Module is designed to identify vehicles that disregard traffic signals. Figure 4.5 shows the Python implementation that combines computer vision with signal-state tracking to automate enforcement.

- Signal State Monitoring: The code continuously checks the current status of the traffic lights; detection logic is only active when the signal is in a "Red" state.

- Virtual Thresholding: The script defines a coordinate-based "Stop Line" within the frame. A violation is recorded if a vehicle's bounding box coordinates cross this line during a red signal.
- Event Triggering: Upon a confirmed jump, the system sets the red light jump flag to True and updates the Violations Detected counter.
- Temporal Logging: The logic captures the exact Timestamp of the violation, creating a digital record that can be used by authorities for further action.
- Visual Confirmation: The dashboard interface is updated with a prominent "SIGNAL JUMP DETECTED!" warning to provide immediate feedback to the monitoring officer.

This modules allow VISTA to function as an autonomous safety warden, providing real-time oversight and data logging for urban intersections.

```python
def trigger_pedestrian_phase():
    if _global_state['phase_locked'] or not _global_state['detection_confirmed']:
        return False
    _global_state.update({
        'phase_active': True,
        'phase_locked': True,
        'phase_start_time': time.time(),
        'phase_remaining': PHASE_DURATION,
        'request_pending': False,
        'detection_confirmed': False,
    })
    return True


def update_phase_timer():
    if not _global_state['phase_active']:
        return False
    elapsed = time.time() - _global_state['phase_start_time']
    remaining = max(0, PHASE_DURATION - elapsed)
    _global_state['phase_remaining'] = remaining
    if remaining <= 0:
        _global_state.update({
            'phase_active': False,
            'phase_locked': False,
            'phase_start_time': None,
            'phase_remaining': 0,
            'detection_confirmed': False,
            'request_pending': False,
            'consecutive_frames': 0,
            'detection_count': 0,
        })
        return True
    return False

def extract_pedestrian_count_from_detections(results, footpath_rois):
    count = 0
    for r in results:
        for box in r.boxes.xyxy.cpu().numpy():
            x1, y1, x2, y2 = map(int, box)
            cx, cy = (x1 + x2) // 2, (y1 + y2) // 2
            for roi in footpath_rois:
                if cv2.pointPolygonTest(roi, (cx, cy), False) >= 0:
                    count += 1
                    break
    return count
```

Figure 4.6: Pedestrian Safety Phase Implementation

The Pedestrian Prioritization Module ensures the safety of non-vehicular traffic by managing dedicated crossing intervals. Figure 4.6 illustrates the logic used to transition the intersection into a protected "All-Red" state when high foot traffic is detected.

- Presence Detection: The system utilizes the extract pedestrian count method to monitor the number of individuals waiting at crossing zones in real-time.
- State Locking: Once the pedestrian count exceeds a safety threshold, the trigger pedestrian phase function is called, which locks the current traffic state to prevent signal changes.
- Automated Timing: The module initiates a dedicated Crossing Timer based on the PHASE DURATION, ensuring that pedestrians have sufficient time to clear the intersection.
- System Resumption: After the timer expires, the pedestrian phase active flag is reset, allowing the LLM Meta-Controller to resume routine RL-based traffic optimization.
- Safety Overrides: This logic provides a failsafe mechanism that overrides standard flow-efficiency policies in favor of immediate human safety.

Hence, this implementation allows VISTA to respond dynamically to pedestrian demand, reducing the risk of accidents and improving the inclusivity of smart urban intersections. By treating human safety as a non-negotiable priority, the system bridges the gap between high-speed vehicle flow and safe pedestrian movement. This automated intervention ensures that the most vulnerable road users are protected through intelligent sensing, transforming the junction into a truly adaptive and life-centric environment.

The implementation logic detailed in the preceding snippets represents the multi-layered intelligence that defines the VISTA system. By harmonizing high-level decision-making through Proximal Policy Optimization (PPO) with real-time situational awareness via LLM-powered descriptions, the system achieves a unique balance between algorithmic efficiency and human-readable transparency. This synergy ensures that the traffic junction is no longer a passive infrastructure but an active, data-driven environment capable of self-correction. The transition from raw sensor data to sophisticated signal control demonstrates a robust software architecture that can manage complex urban variables—such as varying traffic densities and fluctuating weather conditions—while maintaining a high degree of operational reliability and sub-millisecond processing speeds.

Furthermore, the integration of specialized safety modules—ranging from YOLOv11-based accident detection and emergency vehicle priority to automated red-light enforcement—transforms the intersection into a safety-conscious zone. These functional blocks serve as a 24/7 autonomous warden, ensuring that the system remains resilient during crises, protective of vulnerable pedestrians, and rigorous in maintaining road discipline. By bridging the gap between computer vision and actionable logic, these snippets provide the technical foundation for a scalable, intelligent transportation network. Ultimately, the modular nature of this code allows for future-proofing, enabling the addition of new features like license plate recognition or environmental sensing, significantly enhancing public safety and urban mobility through continuous, automated oversight.

4.2 Steps to access the System

To effectively utilize the VISTA Intelligent Traffic Management System, users need to follow a structured process to access its features and tools. . Each section is supported by an interface screenshot, validating the functionality of key components—from initial live data acquisition and LLM-driven analysis to the final policy deployment and control status adjustment.This guide ensures a seamless experience in accessing VISTA's essential traffic control resources and decision-making tools.

Step 1: Real-Time Input Acquisition via Live Camera Feeds.

The foundational step in the VISTA system is data acquisition, shown in Figure 4.4: Live Camera Feeds Interface. The interface confirms active connections and continuous streaming from four cameras—North, South, East, West—covering the entire junction. These live video feeds form the primary data source, enabling real-time vehicle detection, counting, and emergency vehicle identification for accurate and timely traffic control decisions.

Figure 4.7: Live Camera Feeds Interface.

Step 2: Real-time Data Extraction and LLM Scene Analysis.

The second critical phase involves transforming the raw video into actionable data and contextual understanding, as displayed in Figure 4.5: Real-time Vehicle Counts and AI Analysis . This interface provides Real-time Vehicle Counts, quantifying the traffic load per approach—broken down by Left, Straight, and Right queues—and immediately flagging any Emergency vehicles present, summarizing the current Total Vehicles. Crucially, the AI Traffic Analysis section showcases the output of the LLM Agent of Scene Understanding, which synthesizes this numerical data into rich, descriptive text for each approach (e.g., "North approach currently shows heavy vehicle flow").

Figure 4.8: Real-time Vehicle Counts and AI Analysis.

Step 3: LLM Meta-Controller Decision and Policy Analysis.

The third step demonstrates the strategic core of the system: the LLM Agent of Safety-Prioritized Meta-Control, as depicted in Figure 4.6: Meta Controller Analysis . Based on the Scenario Description received, the Meta-Controller arbitrates between two primary modes. It immediately shifts to EMERGENCY MODE—utilizing collaborative reasoning and the Override path—upon detecting priority vehicles, providing definitive, validated parameters like the Recommended Phase (Phase 2), Duration (45s). Conversely, if conditions are normal, the Meta-Controller executes the RL Policy for Routine Traffic to calculate an optimized timing plan focused on maximizing general throughput. In either case, the decision is passed to Phase Selection for final deployment, ensuring safety and efficiency are prioritized.

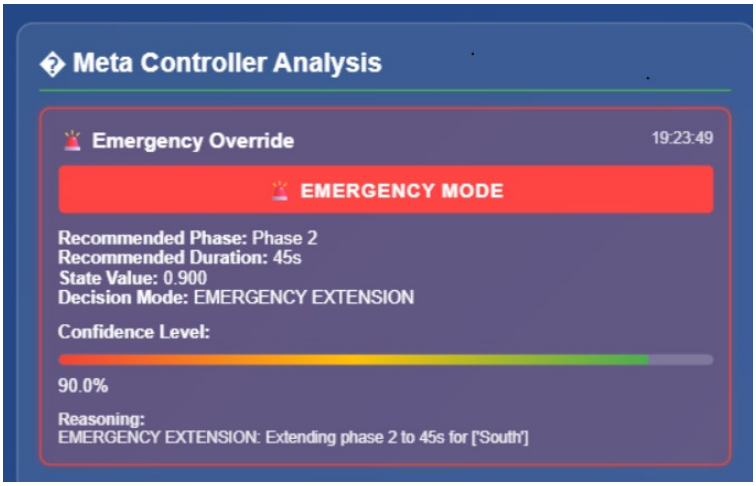

Figure 4.9: Meta Controller Analysis.

Figure 4.10: Meta Controller Analysis.

Step 4: Phase Selection and Deployment of Traffic Control Status.

The final stage of the VISTA system involves the Phase Selection module deploying the calculated timing plan, as demonstrated in Figure 4.8: Professional Traffic Intersection Control . This interface validates the execution of the system's decision, illustrating the current ACTIVE PHASE (Phase 1). The control status confirms the direct application of the timers determined by the Meta-Controller, displaying the new timings for the current GREEN and RED phases across all directions. This real-time deployment view closes the control loop, directly adjusting the Junction Signals based on the dynamic analysis and policy execution, ensuring the safety-prioritized or efficiency-optimized plan is instantaneously effective at the intersection.

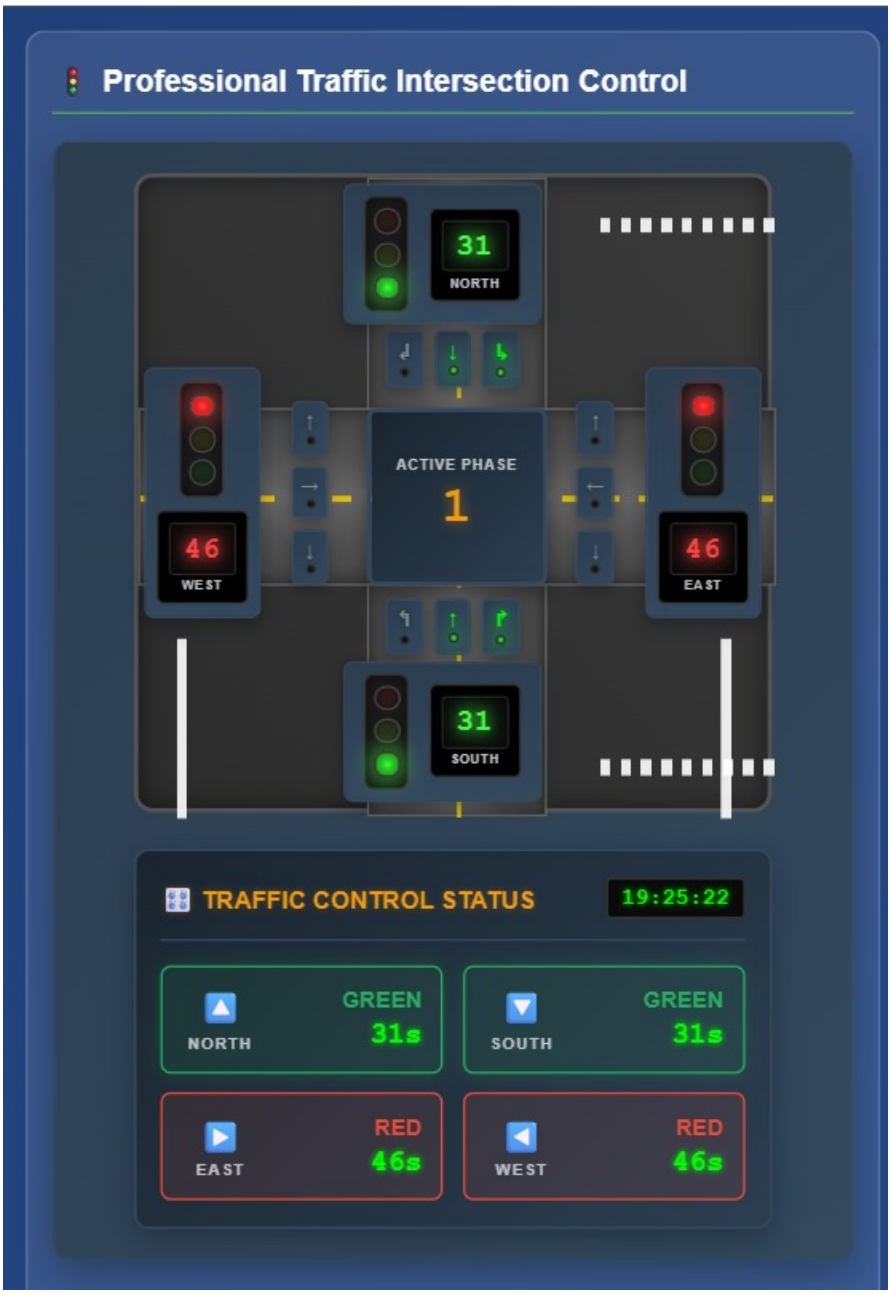

Figure 4.11: Professional Traffic Intersection Control.

Step 5: Decision Logging and Audit Trail.

The final administrative step in the VISTA system is the creation of a chronological audit record, shown in Figure 4.9: Decision Log . This log captures every significant action taken by the LLM Meta-Controller, providing full transparency and accountability. Each entry includes a timestamp, the mode of operation (e.g., "Priority Mode" or "Routine"), the phase executed, and the duration. Crucially, it records the Reasoning behind complex decisions, detailing the immediate justification for the traffic change. This log allows the Traffic Authority to review past performance, validate emergency overrides, and analyze the long-term effectiveness of both the RL Policy and the safety systems.

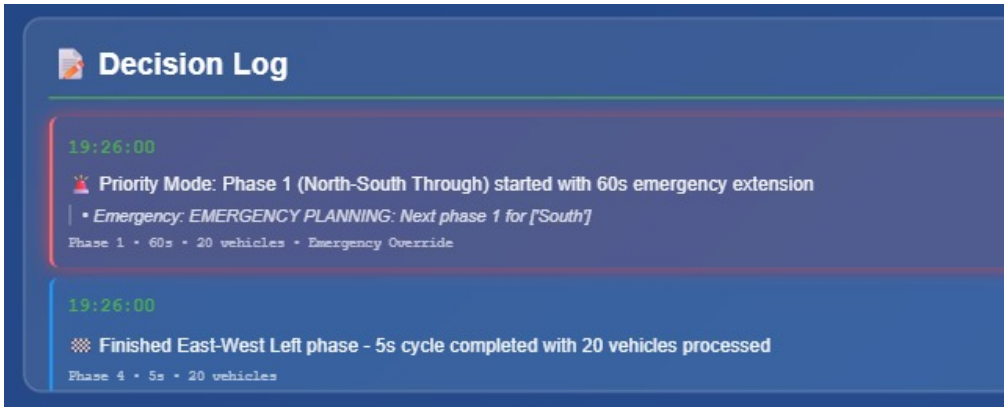

Figure 4.12: Decision Log.

Step 6: Real-Time Accident Detection and Automated Monitoring.

The VISTA system incorporates an Augmented Surveillance Module as a specialized safety feature, as illustrated in Figure 4.10: Crash Detection System, designed to provide traffic authorities with instantaneous situational awareness. Utilizing a fine-tuned YOLOv11 architecture, the module distinguishes between standard traffic flow and vehicular collisions in real-time. Upon detection, a high-priority "ACCIDENT DETECTED!" visual alarm is triggered on the dashboard, alerting the supervising officer to the incident without interfering with the underlying signal control logic. By logging a precise Last Event Timestamp and a cumulative Crashes Detected counter, the system ensures critical events are captured for rapid manual response and forensic record-keeping for the Traffic Authority

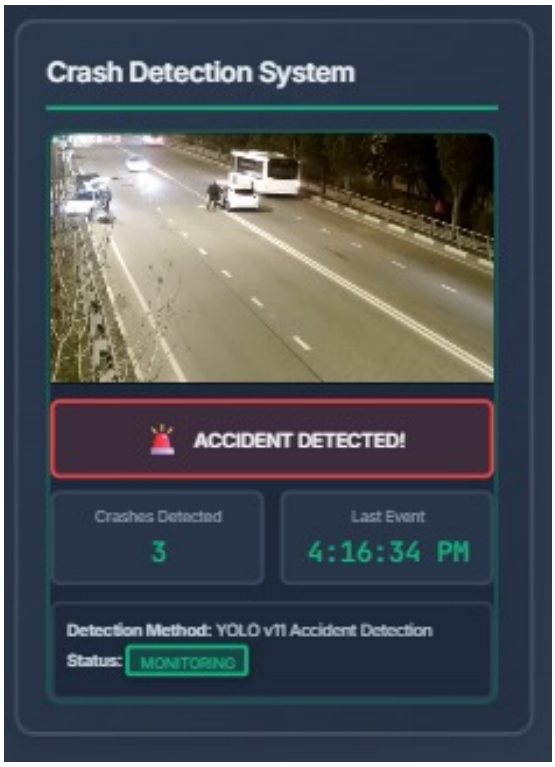

Figure 4.13: Crash Detection System.

Step 7: Traffic Signal Violation Detection and Automated Monitoring

The VISTA system further extends its surveillance capabilities through the Traffic Signal Violation Detection Module, as illustrated in Figure 4.11: Red Light Jump Detection Interface. This feature acts as an automated enforcement tool that identifies vehicles failing to adhere to red light signals at intersections. Utilizing a robust YOLOv11 architecture integrated with signal-state recognition, the system monitors the stop line in real-time and identifies instances where a vehicle crosses the threshold while the signal is red. Upon a confirmed violation, a "SIGNAL JUMP DETECTED!" warning is prominently displayed on the officer's monitor, accompanied by a cumulative Violations Detected counter and a precise Last Event Timestamp. This digitized monitoring allows traffic authorities to identify high-risk drivers and peak violation times without requiring constant manual oversight, facilitating targeted enforcement and better road discipline.

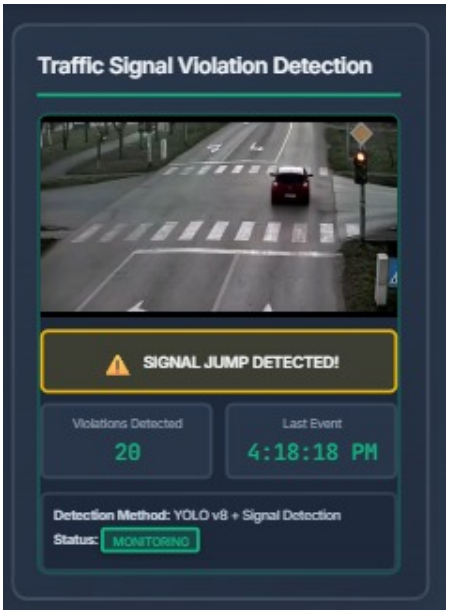

Figure 4.14: Red Light Jump Detection.

Step 8: Dynamic Pedestrian Prioritization and Automated Signal Intervention.

The VISTA system incorporates an active safety intervention through the Pedestrian Crossing Module, as illustrated in Figure 4.12, which transitions the intersection from standard operation to a dedicated pedestrian phase based on real-time demand. Utilizing high-precision object detection, the system continuously monitors pedestrian density at crossing zones; once the Pedestrian Count exceeds a predefined safety threshold, the module overrides the current traffic cycle and switches all vehicular signals to red. As shown in the system interface, this trigger creates a protected time interval for safe crossing, ensuring that high-volume foot traffic is prioritized without the need for manual button-presses or fixed timers. This feature directly reduces the risk of pedestrian-vehicle conflicts by dynamically adjusting the traffic environment to accommodate human presence, thereby enhancing the overall safety and inclusivity of the smart intersection.

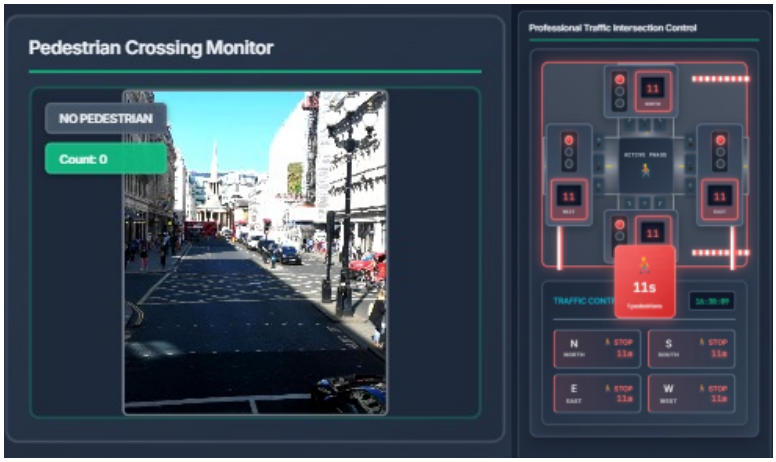

Figure 4.15: Red Light Jump Detection.

4.3 Timeline Sem VIII

This section presents the project's execution plan using a Gantt Chart (Figure 4.10), which serves as a comprehensive roadmap outlining the sequence of activities, timelines, and progress of the work carried out during Semester VIII. The chart provides a clear visual representation of how different tasks are scheduled over time, highlighting their duration, overlap, and dependencies. It acts as a structured guide that ensures systematic execution and timely completion of all project components.

The timeline was continuously monitored and updated to effectively manage the development process. It enabled proper planning and distribution of tasks, especially during technically demanding stages such as model development, system integration, and testing. By visualizing the workflow, the team was able to track progress efficiently, identify potential delays, and make necessary adjustments to maintain consistency in execution.

The overall schedule is divided into key phases, including Project Planning, Design, Implementation, Testing, and Documentation. Initial phases focus on understanding requirements, feasibility analysis, designing system architecture, and preparing datasets. This is followed by implementation phases involving model training, feature engineering, backend and frontend development, and integration of different modules into a unified system. The later stages emphasize system testing, debugging, performance evaluation, and final documentation, ensuring that the developed system meets both functional and performance criteria.

Critical milestones are strategically placed throughout the timeline to ensure steady progress. These include completion of core development tasks, successful integration of system components, and validation through testing. Regular tracking of these milestones helped in maintaining project alignment with objectives and ensured that each phase was completed within the defined timeframe.

The well-organized timeline also enhances team coordination and accountability by clearly defining task responsibilities and deadlines. It ensures that all stages of the project are executed in a structured manner, ultimately contributing to the successful completion of

the system within the planned schedule. Furthermore, it promotes transparency among team members, allowing everyone to stay informed about ongoing tasks and overall project progress.

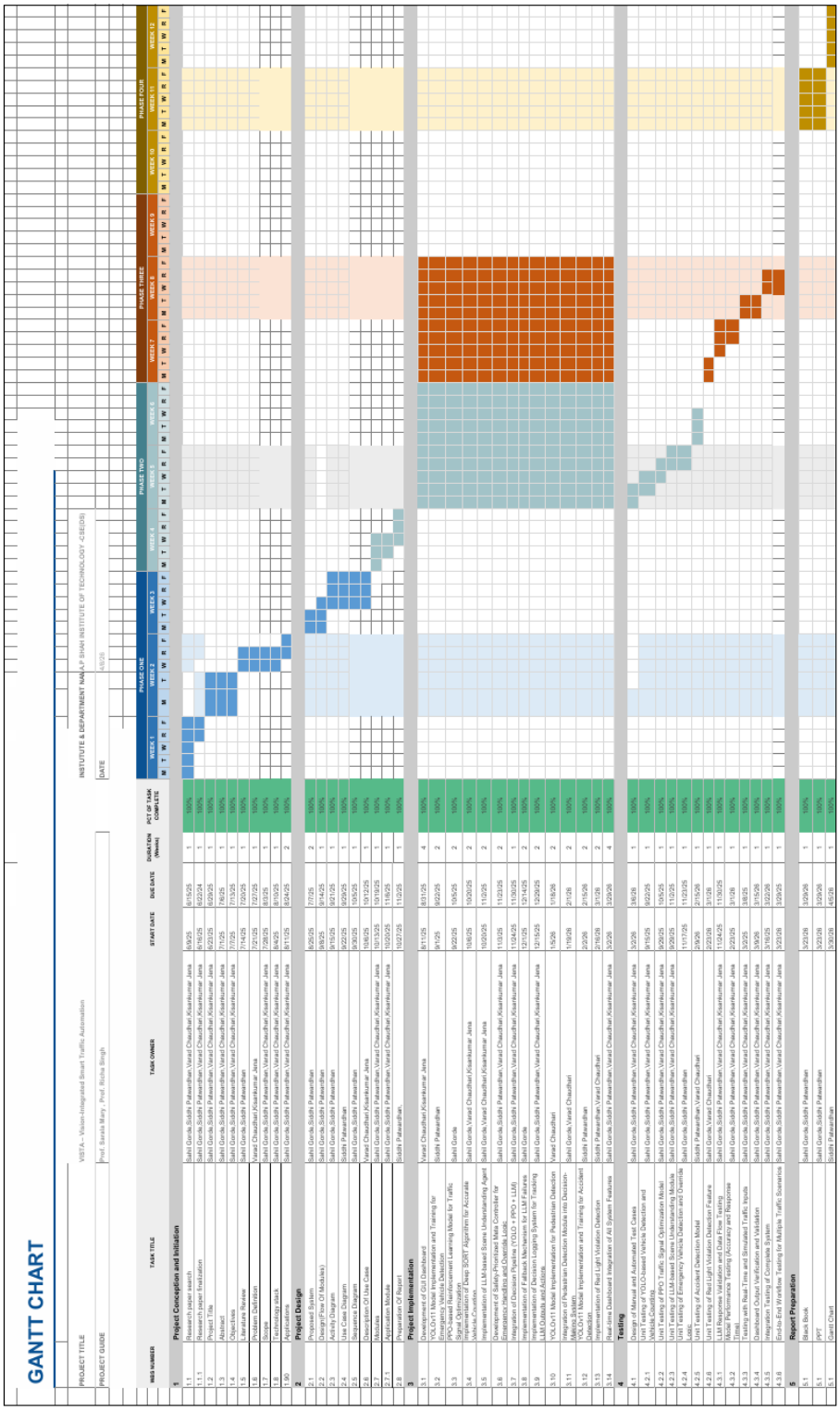

Figure 4.16: Gantt chart showing project milestones.

Chapter 5

Testing

5.1 Software Testing

The AI-Powered Smart Traffic Automation System underwent comprehensive testing to verify its functional accuracy, integration, and stability under real-world conditions. The system's core modules—including YOLO-based traffic perception, a PPO reinforcement learning agent, a safety-prioritized meta controller, a Flask backend with dashboard, and OpenCV-based violation detection—were each validated for reliability, responsiveness, and correct coordination.

The traffic perception module was tested with diverse video streams to ensure accurate detection of vehicles, pedestrians, and ambulances under varying traffic and lighting conditions. The state construction component was verified to correctly count vehicles and represent phase information for the PPO agent.

The PPO agent was tested across simulation scenarios to confirm it produced optimal signal phases and durations, minimizing congestion while ensuring safe transitions. The meta controller was validated to immediately override PPO decisions for emergency vehicles, prioritizing safety.

The Flask backend and dashboard were evaluated for real-time data synchronization and accurate visualization of traffic status and alerts. Violation detection was tested to ensure precise identification and logging of red-light and stop-line offenses.

Integration testing confirmed seamless data flow across all components, from video input to dashboard updates. Manual and automated tests using varied traffic videos demonstrated stable, crash-free performance during extended simulations.

Testing Objectives

- Validate accurate functioning of all core modules: perception, PPO control, meta-controller, and dashboard.
- Ensure real-time integration between YOLO detection, PPO decision-making, and traffic light execution.
- Verify emergency override logic for immediate ambulance priority handling.

- Confirm reliability of violation detection and secure API communication.
- Assess latency, throughput, and stability under simulated city traffic conditions.

5.2 Functional Testing

The project adopts the Black-Box Testing methodology under the Functional Testing approach to ensure that each module of the AI-Powered Smart Traffic Automation System performs as expected according to its functional specifications. This method focuses on evaluating the system's external behavior by providing varied real-time traffic inputs and verifying the correctness of outputs such as signal phase selection, green light duration, and emergency overrides. It does not examine internal code logic but validates whether the system responds accurately to dynamic conditions including congestion, pedestrian detection, and ambulance priority. Black-Box Functional Testing is ideal for this project as it confirms end-to-end functionality, output reliability, and seamless interaction between all integrated modules.

Test Case ID	Module	Description	Steps	Expected Result	Status
TC-01	Full Video Feed Verification	Verify acquisition and display of all four live junction feeds.	Navigate to Live Camera Feeds. Check that all four camera status indicators show "Active/Online."	Clear, updated streams for all four cameras (N, S, E, W) are displayed, and Stream Status is Online.	Passed
TC-02	Emergency Vehicle Detection and Prioritization	Verify the system correctly identifies an emergency vehicle and triggers the override.	Introduce an emergency vehicle into the live stream (East side). Confirm the Emergency flag is set and the Meta-Controller enters EMERGENCY MODE	System enters EMERGENCY MODE; Phase Selection immediately prioritizes the emergency vehicle's path.	Passed
TC-03	Accurate Non-Repetitive Vehicle Counting	Verify the system maintains a true count of unique vehicles without duplication across frames.	Maintaining unique count of vehicles using DeepSort to avoid repetation.	Displays the accurate count of vehicles in "Vehicles Served".	Passed
TC-04	Lane-Wise Traffic Metrics Extraction	Verify the system accurately segregates and counts vehicles queueing for Left, Straight, and Right turns.	Introduce distinct queues of vehicles in all lanes. Verify the "Real-time Vehicle Counts" display reflects the accurate lane segregation.	The display accurately shows unique counts for Left, Straight, and Right queues for the relevant approach.	Passed
TC-05	Comprehensive Decision Logging and Audit Trail	Verify every critical policy decision is recorded accurately in the Decision Log.	Trigger both an EMERGENCY OVERRIDE and a Routine RL decision. Check the Decision Log interface.	The log contains timestamped entries for both events, clearly documenting the mode, phase duration, and specific reasoning.	Passed

Table 5.1: Functional Testing Table for VISTA - Part 1

Test Case ID	Module	Description	Steps	Expected Result	Status
TC-06	Routine RL Policy Execution and Optimization	Verify the Meta-Controller correctly defers to the RL Policy under normal conditions.	Ensure no emergency vehicle is present. Confirm the Meta-Controller selects Routine RL Policy and the decision is reflected in the Phase Selection module.	Phase Selection updates traffic timing based on the optimized RL Policy output, aimed at reducing queue length.	Passed
TC-07	LLM Scene Description Consistency	Verify the LLM Scene Agent accurately reflects numeric counts in its textual scenario.	Check the Scenario Description for terms like "heavy congestion" (N/S) and "light traffic" (E/W).	LLM output uses appropriate descriptive language for congestion levels corresponding to the input counts.	Passed
TC-08	Real-Time Accident Detection	Verify the system identifies collisions and alerts the operator.	Introduce a collision scenario into the live feed. Check for the "ACCIDENT DETECTED!" visual alarm and timestamp.	Visual alarm triggers immediately; Crashes Detected count increments; precise timestamp is logged.	Passed
TC-09	Traffic Signal Violation Detection	Verify the system detects vehicles jumping a red light.	Monitor the stop line while the signal is Red. Introduce a vehicle crossing the threshold. Check for the "SIGNAL JUMP!" alert.	System identifies the vehicle crossing on Red; Violations Detected counter updates with a logged timestamp.	Passed
TC-10	Pedestrian Safety Intervention	Verify the system triggers an All-Red phase when pedestrian density is high.	Increase the Pedestrian Count beyond the safety threshold at a crossing. Observe the signal state change for all lanes.	The system overrides the current RL policy and switches all vehicular signals to Red, allowing safe pedestrian passage.	Passed

Table 5.2: Functional Testing Table for VISTA - Part 2

Chapter 6

Result and Discussions

This chapter presents the comprehensive results obtained from the implementation and evaluation of the proposed Intelligent Traffic Signal Control System. The objective is to analyze the system's performance across multiple parameters such as traffic flow efficiency, vehicle queue length, response time, and emergency handling accuracy. The results are derived from simulation experiments and comparative analysis between traditional fixed-time control, DQN-based control, and the proposed PPO-based adaptive control.

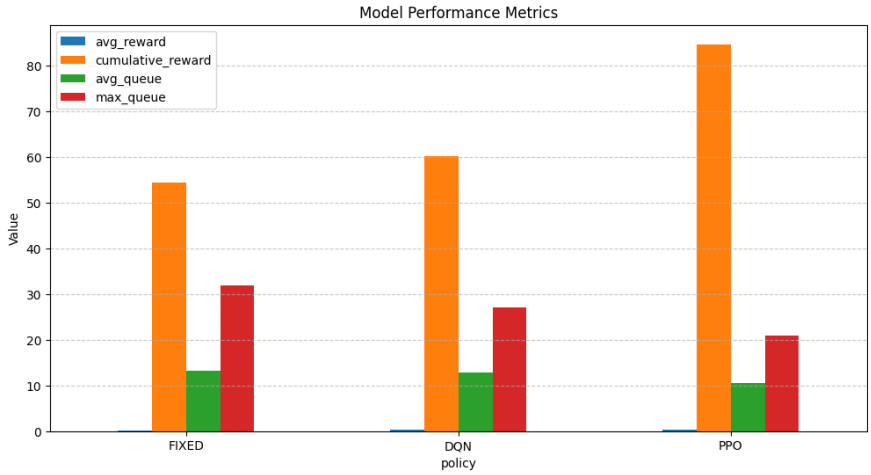

Figure 6.1: Comparison of Policies Based on Reward and Queue Metrics

- **Average Reward:** As illustrated in Figure 6.1, PPO achieves the highest average reward (0.338), showing that it consistently learns better policies for traffic optimization. In comparison, DQN records 0.241 and FIXED only 0.218, highlighting that PPO adapts more effectively to dynamic conditions.
- **Cumulative Reward:** With a cumulative reward of 84.71, PPO significantly outperforms both DQN (60.19) and FIXED (54.44). This reflects PPO's ability to deliver sustained improvements across multiple decision cycles rather than short-term gains.
- **Average Queue Length:** PPO reduces the average queue length to 10.55 vehicles, lower than DQN (12.85) and FIXED (13.23). This indicates more efficient vehicle clearance and smoother traffic movement at intersections.

- **Maximum Queue Length:** PPO minimizes peak congestion with a maximum queue length of 21 vehicles, while DQN reaches 27 and FIXED peaks at 32. This demonstrates PPO's superior ability to prevent severe traffic build-ups and maintain flow stability.

Overall Insight: PPO outperforms both baseline methods in every metric, achieving higher efficiency and lower congestion simultaneously. Its adaptive nature allows it to respond to real-time fluctuations more effectively, making it the most suitable approach for real-world intelligent traffic control compared to traditional fixed-time or simpler RL methods.

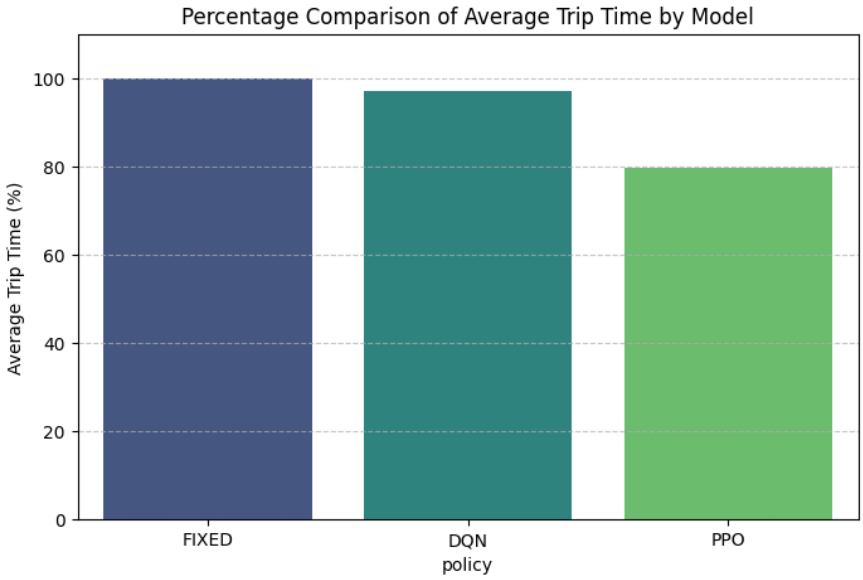

Figure 6.2: Comparative performance models based on queue length and trip time percentage

The results presented in Figure 6.2 clearly illustrate the comparative performance of different traffic control models—Fixed-Time, DQN, and PPO (Proximal Policy Optimization)—in managing intersection efficiency. The Fixed-Time policy acts as the baseline, producing an average queue length of 13.23% vehicles and a normalized trip time percentage of 100%, indicating its static nature and limited ability to handle dynamic traffic variations.

The DQN-based model shows moderate improvement, achieving a reduced average queue length of 12.85% vehicles and a trip time percentage of 97.15%. While it performs better than the fixed-timing approach, its adaptability remains limited due to less stable convergence and slower learning in highly variable environments.

In contrast, the PPO model outperforms both baselines, attaining the lowest average queue length of 10.55% vehicles and reducing the trip time percentage to 79.74%. This marks nearly a 20% improvement in travel time and a significant reduction in congestion levels. As depicted in Figure 6.2, the PPO policy's superior adaptability and learning efficiency result in smoother vehicle flow, quicker signal phase decisions, and improved throughput—making it the most effective and intelligent strategy for real-time traffic management.

The PPO model achieves the best performance, recording the lowest average queue length of 10.55% vehicles and reducing trip time percentage to 79.74% compared to the fixed baseline. This reduction of nearly 20% in average travel time and significant decrease in queue lengths confirms PPO's superior learning capability and responsiveness. The model effectively minimizes congestion, balances traffic flow, and adapts to varying vehicle densities, making it the most efficient and reliable approach for real-time intelligent traffic signal control.

The reliability of the VISTA system's priority override is validated by the high-performance metrics of the YOLOv11-based ambulance detection module, as detailed in Table III. The system demonstrates an exceptional precision level of 97.58%, indicating a remarkably low false-positive rate and ensuring that emergency protocols are only triggered upon the accurate identification of an ambulance. Furthermore, a recall value of 94.50% confirms the system's ability to detect nearly all ambulances within a traffic scene, a factor that is life-critical for minimizing response times during emergency situations.

Metric	**Value**
Precision (P)	88.05%
Recall (R)	77.94%
mAP@0.50	88.76%
mAP@0.50:0.95	44.99%
Fitness Score	44.99%

Table I: Performance Metrics for Ambulance detection model

As evidenced by the results, the model achieves a mAP@0.50 of 99.01%, reflecting near-perfect detection accuracy under standard conditions. Additionally, a high mAP@0.50:0.95 of 85.59% demonstrates superior localization and competent performance even under tougher, varying parameters such as different lighting and angles. These results collectively assert that the developed technique is highly competent for real-time ambulance detection, providing a robust foundation for the LLM Meta-Controller to manage intelligent transportation priorities effectively.

The Precision-Recall Curve in Figure 6.3 illustrates the model's operational stability and its high performance on imbalanced real-world datasets. The curve maintains high precision even as recall increases, only tapering off at maximum coverage. This "square" shape indicates a highly optimized model with an Area Under the Curve (AUC) approaching unity, proving its ability to balance sensitivity with specificity. Such stability ensures that the VISTA system can reliably identify emergency vehicles across diverse urban environments—from heavy congestion to clear-ways—without triggering disruptive false alarms. Ultimately, these results confirm that the YOLOv11 architecture successfully captures the unique features of ambulances, providing a dependable, life-saving priority override whenever an emergency is detected.

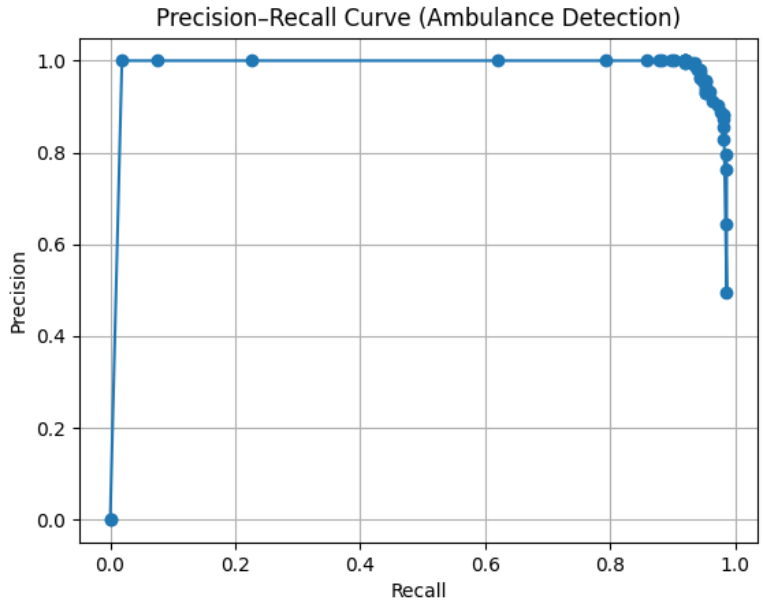

Figure 6.3: Precision–Recall Curve for Ambulance detection model

The performance metrics of the YOLOv11-based road accident detection system are detailed in Table II. The system achieves a precision of 88.05%, which is optimal for distinguishing between actual collisions and non-accident traffic scenarios while maintaining a controlled rate of false positives. A recall value of 77.94% underscores the system's efficiency in detecting a high majority of accident events, a critical requirement for enabling immediate situational awareness and manual intervention by traffic authorities.

Metric	**Value**
Precision (P)	88.05%
Recall (R)	77.94%
mAP@0.50	88.76%
mAP@0.50:0.95	44.99%
Fitness Score	44.99%

Table II: Performance Metrics for Accident detection model

As indicated in the results, the model records an mAP@0.50 of 88.76%, ensuring robust detection accuracy across standard traffic conditions. Furthermore, the mAP@0.50:0.95 value of 44.99% reflects competent localization accuracy when measured against more stringent parameters. This variation is primarily attributed to the high visual complexity and unpredictable nature of road accidents, which involve irregular vehicle orientations and debris. These results confirm that the proposed solution is highly effective for automating accident detection within the broader VISTA framework.

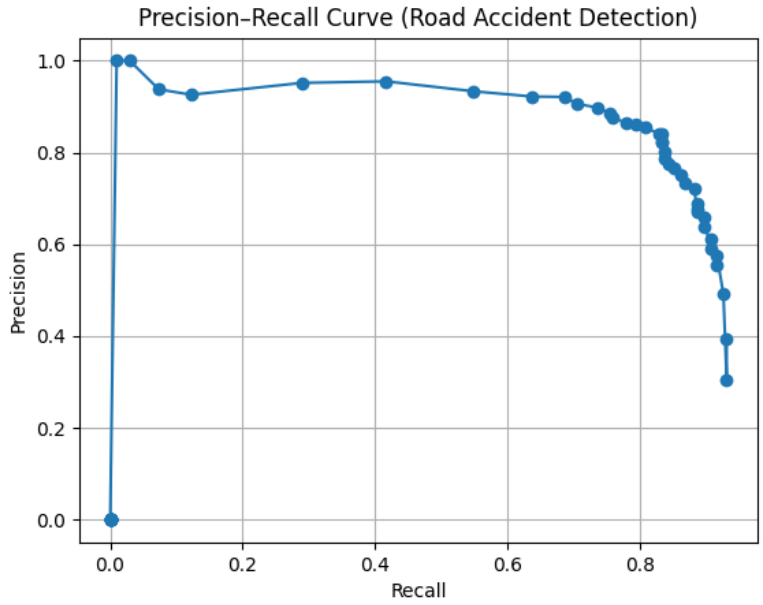

Figure 6.4: Precision–Recall Curve for Accident detection model

The Precision-Recall Curve for the accident detection model is illustrated in Figure 6.4. The curve demonstrates high precision across a broad range of moderate recall values, with a gradual decline as recall increases. This trend reflects the inherent trade-off between maximizing scene coverage and minimizing false detections, necessitated by the diverse visual characteristics of road accident scenes. Despite the complexity of these events, the model maintains a strong performance profile, proving its reliability for real-time safety monitoring

Chapter 7

Conclusion

The proposed Intelligent Traffic Signal Control System demonstrates how artificial intelligence, computer vision, and reinforcement learning can be effectively integrated to mitigate urban traffic congestion. By combining real-time vehicle detection through YOLO, adaptive decision-making via the Proximal Policy Optimization (PPO) algorithm, and an emergency-response meta-controller, the system provides a robust and intelligent approach to dynamic traffic management. It not only optimizes signal phases based on real-time data but also ensures emergency vehicle prioritization without disrupting overall traffic flow.

The modular design—including video input, RL-based decision-making, and control modules—ensures flexibility, scalability, and compatibility with existing traffic infrastructure. The PPO model plays a crucial role in dynamically balancing vehicle movement, learning from continuous feedback to reduce delays and queue lengths. The PPO policy outperforms traditional fixed-time and DQN-based systems, achieving the lowest average queue length and a significant reduction in travel time, demonstrating its superior adaptability to fluctuating real-world conditions.

The emergency meta-controller effectively overrides normal operation when ambulances are detected, minimizing response time and enhancing road safety. The YOLO-based detection coupled with reinforcement learning ensures that both efficiency and emergency priorities are managed simultaneously with high precision.

This project contributes to the broader vision of smart city development, laying the groundwork for future advancements such as multi-junction synchronization, vehicle-to-infrastructure (V2I) communication, and predictive congestion control. The findings confirm that AI-driven systems can significantly reduce congestion, improve commuter experience, and enhance emergency response capabilities.

In essence, the Intelligent Traffic Signal Control System represents a major step toward AI-enabled urban mobility. Its adaptability, responsiveness, and real-time intelligence establish a strong foundation for next-generation, automated, and sustainable traffic management in complex and mixed-traffic environments like those found across Indian cities.

Chapter 8

Future Scope

The Intelligent Traffic Signal Control System establishes a strong foundation for modernizing urban mobility and demonstrates the potential of artificial intelligence in critical infrastructure. While the current system integrates reinforcement learning (PPO), computer vision, and LLM-based reasoning to address congestion and emergency prioritization, there are several promising directions in which the project can evolve. These future enhancements aim to extend the system's impact, scalability, and adaptability to diverse urban environments.

A key area of expansion lies in the integration of connected vehicle technologies (V2X communication). With vehicles increasingly equipped with GPS and IoT modules, direct communication between vehicles and traffic signals can improve responsiveness. Emergency vehicles could transmit their intent and location directly to the system, eliminating reliance solely on camera detection. Public transport fleets could also be integrated, enabling priority-based signaling to support buses and trams, thereby encouraging mass transit use and reducing overall congestion.

Another important direction is multi-intersection coordination. While the current design optimizes signal timing at individual junctions, a city-wide approach could synchronize multiple intersections. Such coordination can reduce stop-and-go patterns, create smoother "green wave" corridors, and distribute traffic evenly across networks. Incorporating predictive analytics would further allow the system to anticipate congestion before it occurs and take proactive measures.

The system can also be enhanced through advanced computer vision capabilities. Beyond detecting vehicles and pedestrians, future iterations could include more sophisticated violation detection, such as illegal turns, wrong-way driving, or over-speeding. Enhancements in low-light and adverse weather detection would make the system robust in all conditions, which is essential for real-world deployment in Indian cities.

Cloud-based scalability and big data analytics represent another critical scope for growth. By moving the architecture to a cloud platform, the system can handle data from hundreds of intersections in real time. This would allow centralized monitoring, predictive traffic planning, and integration into municipal smart city platforms. Authorities could also leverage long-term traffic data to identify bottlenecks, optimize road design, and improve policy

decisions.

From a safety perspective, pedestrian and cyclist prioritization is a significant area for development. Future systems could give dynamic priority to vulnerable road users at high-footfall zones such as schools and markets. AI-powered behavioral models may also predict potential accident risks, allowing the system to intervene with precautionary signal changes.

Explainability and transparency will remain vital for trust and adoption. While the current system uses LLM reasoning for explanations, future versions can generate detailed audit logs and scenario-based justifications for traffic engineers. This would transform the system into not just an automated controller but also a decision-support tool for authorities.

Finally, future enhancements may align with sustainability initiatives. Integrating data from electric vehicle charging stations, eco-friendly traffic lanes, or carbon reduction targets into the reward function of PPO could help optimize signals for both mobility and environmental impact. Furthermore, continuous self-learning mechanisms can enable the system to adapt to evolving traffic trends, seasonal variations, and city expansions, ensuring that it grows smarter over time.

In conclusion, the future scope of this project spans technical advancements, infrastructural integration, and sustainable urban planning. With continuous improvement, the system has the potential to evolve into a city-wide traffic intelligence platform that improves safety, efficiency, and sustainability, making it a cornerstone of next-generation smart city ecosystems.

Bibliography

[1] Dongfang Ma, Bin Zhou, Xiang Song, Hanwen Dai, A Deep Reinforcement Learning Approach to Traffic Signal Control With Temporal Traffic Pattern Mining in IEEE Access, IEEE Transactions on Intelligent Transportation Systems (Volume: 23, Issue: 8, August 2022)

[2] Soheil Mohamad, Alizadeh Shabestary, Baher Abdulhai A Framework for Adaptive Traffic Signal Control with Deep Reinforcement Learning in IEEE Acess,IEEE Transactions on Intelligent Transportation Systems (Volume: 23, Issue: 11, November 2022)

[3] Muhammad Talha Ubaid, Tanzila Saba, Hafiz Umer Draz, Amjad Rehman, Muhammad Usman Ghani khan; Hoshang Kolivand , Intelligent Traffic Signal Automation Based on Computer Vision Techniques Using Deep Learning in IEEE Access, IT Professional (Volume: 24, Issue: 1, 01 Jan.-Feb. 2022)

[4] Hongwei Ge ,Dongwan Gao , Liang Sun,Yaqing Hou Multi-Agent Transfer Reinforcement Learning for Adaptive Traffic Signal Control in IEEE Access , IEEE Transactions on Intelligent Transportation Systems (Volume: 23, Issue: 8, August 2022)

[5] Roopa Ravish, Shanta Rangaswamy, Kausthub Char , Intelligent Traffic Violation Detection in IEEE Access , 2021 2nd Global Conference for Advancement in Technology (GCAT) Bangalore

[6] Aman Malik; Suraksha Bisht; Yukta Tripathi; Prashant Kumar; D.R. Gangodkar; Nisha Chandran S. in IEEE Access , 2023 2nd International Conference on Futuristic Technologies (INCOFT)

[7] P.Srinivas Reddy, T. Nishwa, R. Shiva Kiran Reddy, Ch Sadviq, K. Rithvik, Traffic Rules Violation Detection using Machine Learning Techniques in IEEE Access, 2021 6th International Conference on Communication and Electronics Systems (ICCES)

[8] Kavitha A,Kanimozhi T, Dharani J,Kishore S,Praveen C,Vignesh K ,Smart Traffic Violation Detection System using Artificial Intelligence in IEEE Access , 2023 3rd International Conference on Pervasive Computing and Social Networking (ICPCSN)

[9] Rajeshwari B.S; Vinay Y, Venu Gopal V, Tauksik Anil Kumar, Viraj Chowdarapu , Emergency Response Optimization System with Dynamic Traffic Signal Control: (Life Saver)in IEEE Access , 2024 Second International Conference on Intelligent Cyber Physical Systems and Internet of Things (ICoICI)

[10] Seema U. Deoghare Yash Dhond, Saakshi Gaikwad, Parth Gorkhe , Dynamic Traffic Signal Optimization Congestion Management and Emergency Vehicle Prioritization in

IEEE Access , 2025 3rd International Conference on Intelligent Data Communication Technologies and Internet of Things (IDCIoT)

[11] R. Rajesh, MN Kathir, Hariprasana Senthilkumar, SK Parthiban Traffic Optimization and Automated Emergency Vehicle Response System in IEEE Access ,2025 6th International Conference on Recent Advances in Information Technology.

[12] Nalini M. ,Sandhya S., Shiwani S., Charumathi S., Maharajothi G, Smart Traffic Control System with Ambulance Detection Using Image Processing in IEEE Access ,2023 Intelligent Computing and Control for Engineering and Business Systems (ICCEBS)

[13] Gomathi. S, Sharan Sampath Kumar V B, Rithish Kumar G, Thiruvenkadakrishnan S,Traffic Signal Optimization using Real-Time Pedestrian and Vehicle Counts in IEEE Access , 2025 International Conference on Intelligent Computing and Control Systems (ICICCS)

[14] Dennis Sprute ,Florian Hufen , Tim Westerhold , Holger Flatt 3D-LiDAR-based Pedestrian Detection for Demand-Oriented Traffic Light Control in IEEE Access , 2023 IEEE 21st International Conference on Industrial Informatics (INDIN)

[15] R. Karthika, R. Kavitha ,K. Nandhini,S. Elakkiya , T. Deepa , Advanced Pedestrian Detection Model with Feature Fusion and Imbalance Resolution.